

claude-windows / c7679509-ee2b-4443-b240-0b3b1b1a7291

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\c7679509-ee2b-4443-b240-0b3b1b1a7291\subagents\workflows\wf\_a2b97cff-dcd\agent-af15b5a3acbc06d09.jsonl`
- SHA-256: `5d9fd2c61423d93cdf83b8d632814484bdd8a3c198b85292f1b10ba49ad889f5`
- Source modified: `2026-06-21T13:37:57+00:00`
- Imported at: `2026-07-05T16:48:23+00:00`
- project: `wf\_a2b97cff-dcd`
- session_id: `c7679509-ee2b-4443-b240-0b3b1b1a7291`

Transcript

1. user (2026-06-21T13:34:48.458Z)

PROJECT: a local, AI-powered badminton/racket-sports HIGHLIGHT INDEXER + rally detector.
Engine repo (PRIMARY): C:/Users/avidu/Projects/badminton-highlight-indexer (Python, FastAPI + SQLite + ffmpeg + GPU CV/AI; paid Gemini = cloud AI).
Sibling repos: C:/Users/avidu/Projects/khelsutra (the web/ React SPA + site/ marketing), C:/Users/avidu/Projects/rally-annotator (VLC plugin, MIT), C:/Users/avidu/Projects/sports-data-collector, C:/Users/avidu/Projects/sports-obsreport.

GOAL of the plan we are building: bake the engine into a DOWNLOADABLE, "batteries-included" Python APP that a user can download and run WITHOUT sourcing the repo (no git clone / build-from-source). It must spin up a local WEB SERVICE that runs (a) INFERENCE (video -> rallies -> highlight reel), (b) TRAINING (BYO model on their own labelled data), and (c) RALLY ANNOTATION if needed -- runnable BOTH locally and against CLOUD services. Preserve every already-completed end-to-end CUJ.

KEY FACTS the orchestrator already verified:

- pyproject.toml: hatchling build, namespace package (no backend/__init__.py), packages=["backend","training"], console script: indexer = backend.main:main. requires-python >=3.10.
- torch/torchvision are DELIBERATELY NOT in dependencies -- their CUDA/CPU wheels need --index-url (PEP621 cannot express). Optional extras: gpu(empty, docs-only), auth(PyJWT), reporting(git+ssh obsreport), storage-s3(boto3), storage-gcs(google-cloud-storage). google-genai is a hard dep.
- Server today: python -m backend.main --server --port 8000 . Worker: python -m backend.worker {infer|train} . CLI single-shot: python -m backend.main --video-path ... --segmenter {gemini|yolo_hybrid|wasb_hybrid|motion}.
- THE HARD PACKAGING PROBLEM: native WASB shuttle model runs in a SEPARATE Python env (conda, torch 1.11+cu113) while the main engine uses torch 2.6+cu124. ffmpeg/ffprobe are system deps. Model weights (wasb_badminton_best.pth.tar) live in a GCS bucket.
- Decoupled-compute SEAM already built (M1-M8 merged, default-OFF): deployment.profile + compute_target on AppConfig; backend/config/presets.py (PROFILE_PRESETS: truly-local / cloud-serving / cpu-mock); backend/storage/* (local/gcs/s3/gdrive StorageBackend registry); backend/compute/* (ComputeBackend registry, CloudRunCompute dispatches a Cloud Run Job); backend/config/startup.py per-profile checks; backend/billing.py cost ledger.
- The web SPA lives in a DIFFERENT repo (khelsutra/web, Vite/React/TS/Tailwind, i18n en+kn+hi). The engine has a likely-DEAD backend/api/static. Bundling the UI is a cross-repo concern.
- Licensing guardrail (NON-NEGOTIABLE): every dependency -- code, data, AND weights -- must be MIT/Apache-2.0/BSD only. Paid LLMs = inference/serving only, never training data. Any NEW packaging dependency must also be license-clean.
- A prior docs/DESKTOP_APP_PLAN.md exists but was SUPERSEDED by owner decision D10: deliver a "self-hosted webserver + UI the user spins up" (NOT a desktop app). khelsutra/docs/LOCAL_APPLIANCE_ARCHITECTURE.md is the live appliance design.

Always ground claims in file:line or a doc path. Read the actual files; do not assume.

AREA: rally annotation + BYO training. READ: rally-annotator repo (README, how the VLC plugin installs + its CSV output schema), khelsutra/docs/ANNOTATE_AND_TRAIN.md (if present), engine backend/worker train CLI (backend/worker/___main___py cmd_train), how labelled CSVs are ingested (gt_loader / stub_runner), the ship-gate (backend/eval harness gate_verdict / no_paid_llm_training_labels / label_source). Answer: what "rally annotation if needed" means in-app (can annotation be in-browser, or does it stay the external VLC plugin?); the exact train workflow + its prereqs (GPU? the dual env?); the licensing ship-gate that refuses commercial models trained on paid-LLM labels; what a packaged app must include to offer annotate->train.

2. assistant (2026-06-21T13:34:51.332Z)

I'll investigate the rally annotation + BYO training area. Let me start by pulling the repos and reading the relevant files.

3. user (2026-06-21T13:34:57.771Z)

Already up to date.

---ENGINE---

Already up to date.

---KHELSUTRA---

Already up to date.

4. user (2026-06-21T13:34:57.880Z)

```
C:\Users\avidu\Projects\rally-annotator\docs\ENDING_REASONS.md
C:\Users\avidu\Projects\rally-annotator\docs\CSV_FORMAT.md
C:\Users\avidu\Projects\rally-annotator\README.md
C:\Users\avidu\Projects\rally-annotator\CHANGELOG.md
C:\Users\avidu\Projects\rally-annotator\test\README.md
```

5. assistant (2026-06-21T13:35:00.797Z)

Let me read the rally-annotator docs and the engine training/eval files in parallel.

6. user (2026-06-21T13:35:01.309Z)

```
1      # Rally Annotator
2
3      [![tests](https://github.com/avidullu/rally-
annotator/actions/workflows/ci.yml/badge.svg)](https://github.com/avidullu/rally-
annotator/actions/workflows/ci.yml)
4
5      A tiny VLC plugin to hand-label rally start/end + point-stop reason while you
watch a match,
6      for net-separated racquet sports ? badminton, tennis, table tennis, pickleball,
padel.
7
8      You watch the video in VLC, pause/scrub freely, and click Mark START / Mark END
(with a
9      reason). It writes one CSV row per rally ? clean ground-truth labels for
training/evaluating rally
10     (point) segmentation models, or just for cutting highlights.
11
12     ```
13     rally_number,start_time,end_time,ending_reason,sport,shots_count
14     1,8.800,11.500,winner,badminton,9
15     2,24.389,46.589,unforced_error,badminton,21
16     ```
17
18     Times are decimal seconds; `shots_count` is an optional rally shot/stroke count
(blank if you skip it).
19     See [docs/CSV_FORMAT.md](docs/CSV_FORMAT.md).
20
21     ## Why
```

22 Labeling rally boundaries is the slow, expensive prerequisite for any rally-detection
model. Most tools
23 make you transcribe timestamps by hand. This lets a rater do it ****inside the player they
already use****,
24 pausing to the exact frame before each mark ? turning "watch a match" into "produce
golden labels."
25
26 **## Install**
27 1. Copy `vlc/rally\_annotator.lua` into your VLC Lua ****extensions**** folder:
28 - ****Windows:**** `%APPDATA%\vlc\lua\extensions\
29 - ****macOS:**** `~/Library/Application Support/org.videolan.vlc/lua/extensions/
30 - ****Linux:**** `~/.local/share/vlc/lua/extensions/
31 (create the `extensions` folder if it doesn't exist)
32 2. In VLC: ****Tools ? Plugins and extensions ? Reload extensions**** (or just restart VLC).
33 3. Enable it from the ****View**** menu ? ****Rally Annotator****. A small dialog opens and
stays open while you watch.
34
35 Requires ****VLC 3.0.x****. (VLC 4.0 changed the Lua input/listener API; targeting 3.x for
now.)
36
37 **## Use**
38 ****Playback is built in**** ? the ****Back 5s · Play / Pause · Fwd 5s**** row drives the VLC
player from the
39 annotation window, so you can pause, label, and resume without ever switching to the
main VLC window. ****Play / Pause****
40 is a single toggle (it resumes when paused, pauses when playing).
41
42 1. Pick the ****Sport**** (it stays set across rallies).
43 2. When a rally begins, click ****Mark START**** ? it snapshots the current playback time
into the ****Start**** field
44 (pause/scrub first for frame accuracy; you can also edit the field by hand).
45 3. When the rally ends, click ****Mark END**** (fills the ****End**** field). This ***arms*** the
rally; nothing is written yet.
46 4. Choose the ****Ending reason**** (sits between ****Mark END**** and ****Save Rally****; defaults
to ****`unknown`****), optionally
47 type a ****Number of shots**** (the rally's shot/stroke count ? leave blank to skip),
then click
48 ****Save Rally**** to write one CSV row. The reason ****resets to `unknown`** after every
save (and the shots field
49 clears too) ? so neither silently reuses the previous rally's value, and you can pick
the same reason on
50 consecutive rallies. Use the ****Next rally #**** field to resume/insert numbering
anywhere.
51 5. ****Recent rallies**** list: select a row, then ****Edit selected**** (loads it back into the
fields to fix
52 start/end/reason/sport ? ****Save changes****) or ****Delete selected****. ****Undo last****
removes the most recent row ?
53 the button shows which one, e.g. ****Undo last (#7)****, and the status panel shows that
row's details ? or clears an
54 in-progress mark, or cancels an edit.
55 6. ****Help**** button ? usage + an [ending-reason decision guide](docs/ENDING_REASONS.md),
rendered inside the dialog.
56
57 ****Output:**** `.rallies.csv` next to the video (falls back to your home dir if
the path can't be
58 resolved). Re-opening the extension ****continues**** rally numbering from the existing file
? no duplicate IDs.
59
60 ****Resuming a half-finished video:**** labels are saved to that CSV as you go. Open the
****same video**** and enable the
61 extension and it ****reloads your existing rallies**** (shown in the Recent list) and
continues numbering ? so you can
62 stop and pick up later. Easiest flow: ****open the video first, then enable the
extension.**** If you enable it ***before***
63 the video is loaded, that's fine too ? the moment you click ****Mark START**** (the video is

```

playing by then) the tool
64     switches to that video's own `<video-stem>.rallies.csv` and loads any rallies already
saved for it, so nothing is
65     written to the home-dir fallback by mistake. If you switch videos with the dialog
already open, click **Refresh** to
66     point at the current video and load its rallies. (Playback *position* isn't restored ?
scrub to where you stopped.)
67
68     > **Where's the help / "About" text?** The empty "About ?lua" box under **Tools ?
Plugins and extensions ?
69     > Add-ons** is VLC's own add-on info dialog; its body ("Lua script") is a constant baked
into VLC and **can't be
70     > set by an extension**. Use the **Help** button inside the Rally Annotator dialog
instead. (The descriptor's
71     > description does show in the *Active Extensions* tab's **"More information"*** dialog,
and the *Active Extensions*
72     > list shows the title with its version, e.g. **`Rally Annotator v1.6.3`**, so you can
confirm which build is loaded.)
73
74     ## Sports & taxonomy
75     Net-separated racquet sports share a forced/unforced-error point-stop taxonomy, so one
tool covers them all:
76
77     | Sport | Notes |
78     |---|---|
79     | badminton | the reference sport |
80     | tennis | service_fault = fault/double-fault; let supported |
81     | table_tennis | let (net serve) supported; fast cadence |
82     | pickleball | service_fault for faults; treat "let" per local rules |
83     | padel | net-separated; rally boundaries as in tennis |
84
85     `ending_reason` ? `{unknown, winner, forced_error, unforced_error, service_fault, let,
other}` ? a shared
86     vocabulary across these sports. Keep to these values for clean downstream aggregation.
87
88     ### What the ending reasons mean
89     `unknown` is the **default** (the field resets to it after every save); every other
reason **except `winner`** is
90     charged to the side that **lost** the rally.
91
92     | Reason | When the rally ended because? |
93     |---|---|
94     | `unknown` | **default** ? not classified yet. A save with no reason picked records
`unknown` (never the previous rally's reason); set a specific reason when you can. |
95     | `winner` | the last shot landed **in** and went unreturned (opponent couldn't reach it
/ only waved). A clean ace counts here. |
96     | `forced_error` | the loser **missed** (out or into the net) while **under pressure** ?
stretched, rushed, jammed, handling the opponent's pace/spin/depth. |
97     | `unforced_error` | the loser **missed a routine shot** they had time **and** position
to make, with little or no pressure. |
98     | `service_fault` | the point ended **on the serve** ? into the net, out of the service
box, illegal action/foot fault, or a double fault. |
99     | `let` | the rally is **replayed** under the rules, no point scored (e.g. a
tennis/table-tennis serve net-cord that's otherwise good, outside interference). |
100    | `other` | none of the above ? occluded footage, injury/retirement, penalty, hindrance.
Use sparingly. |
101
102    **The cases people ask about:** a ball/shuttle landing **out** is the *hitter's* error ?
`forced_error` only if they
103    were under pressure, otherwise `unforced_error` (it's **never** automatically forced,
and never a winner for the
104    opponent). **Into the net** during a rally is the same; a **serve** into the net is
`service_fault`. When unsure
105    forced-vs-unforced, **default to `unforced_error`**. See the full decision guide with
examples and per-sport rules

```

```

106     in **[docs/ENDING_REASONS.md](docs/ENDING_REASONS.md)** (also available via the in-
plugin **Help** button).
107
108     ## Roadmap
109     - [ ] Live-test the v1.6 dialog in VLC (single Play / Pause toggle, optional Number of
shots, two-step Save,
110         `unknown`-default reason, editable times, Next rally #, Recent-rallies
Edit/Delete) across all five sports.
111     - [ ] Optional per-sport reason presets / hotkeys.
112     - [ ] Alternative front-ends for power users / remote raters: a `python-vlc` + Tk/Qt app
with global keyboard
113         shortcuts (S/E/1?6/U), and a zero-install HTML5 `` page that exports the
same CSV.
114     - [x] Optional `shots_count` column (v1.6). Further extra columns (e.g. server/receiver)
still TBD behind a toggle.
115
116     ## Tests
117     The dialog logic has a headless test suite that stubs VLC's `vlc` API (widgets +
playback), loads the real
118     extension, drives its callbacks, and asserts the results ? run with **Lua 5.1** (the
interpreter VLC 3.x embeds):
119
120     ```bash
121     lua5.1 test/dialog_test.lua          # exit 0 = pass; covers reason reset, shots column,
numbering, undo, help toggle, Play/Pause toggle
122     ```
123
124     It covers all the logic but not VLC's GUI rendering or real playback (there's no
headless VLC UI automation) ? those
125     are checked by loading in VLC with `vlc -vv --file-logging` (must scan with no `Error
loading` line) plus a manual
126     click-through. See [test/README.md](test/README.md).
127
128     ## Contributing
129     Issues and PRs welcome ? especially live test reports per sport/OS and small UX fixes.
The plugin is a single
130     self-contained Lua file (`vlc/rally_annotator.lua`) using only documented VLC 3.x Lua
APIs; please keep
131     `test/dialog_test.lua` green (`lua5.1 test/dialog_test.lua`) and add a case for new
dialog logic.
132
133     ## License
134     MIT ? see [LICENSE](LICENSE).
135

```

7. user (2026-06-21T13:35:01.784Z)

```

1     # Output CSV format
2
3     One row per rally. Header is written once when the file is created.
4
5     ```
6     rally_number,start_time,end_time,ending_reason,sport,shots_count
7     1,8.800,11.500,winner,badminton,9
8     2,24.389,46.589,unforced_error,badminton,21
9     3,49.183,54.683,let,badminton,
10    ```
11
12    | Column | Type | Notes |
13    |---|---|---|
14    | `rally_number` | int | 1-based, monotonic; continues across re-opening the same CSV |
15    | `start_time` | float | rally start, **decimal seconds** from the video start |
16    | `end_time` | float | rally end, decimal seconds (always `> start_time`) |
17    | `ending_reason` | enum | `unknown` (default) / `winner` / `forced_error` /
`unforced_error` / `service_fault` / `let` / `other` ? see the decision guide in

```

```

[ENDING_REASONS.md](ENDING_REASONS.md) |
18 | `sport` | enum | `badminton` / `tennis` / `table_tennis` / `pickleball` / `padel` |
19 | `shots_count` | int | **optional** ? number of shots/strokes in the rally. **Blank**
when not entered (as in row 3 above); added in v1.6. Older CSVs without this column still load.
|
20
21 Notes:
22 - **Decimal seconds**, not `mm:ss` ? downstream loaders parse floats.
23 - Rows are appended live; **Undo last** truncates the last row (one level).
24 - Consumers should read by **column name** (the header is stable but column *order* may
evolve; extra
25 columns may be added later). Most CSV `DictReader`-style loaders already tolerate
extra columns.
26
27 ## Ingesting the labels
28
29 This CSV is intentionally compatible with common rally-segmentation tooling that reads
30 `rally_number,start_time,end_time` by header name and drops rows where `end <= start`:
31
32 ```bash
33 # Generic: any tool that takes a labels CSV of start/end seconds.
34
35 # Example ? score a detector against these labels, or slice clips by them
36 # (badminton-highlight-indexer loaders read --labels <this.csv> directly):
37 python -m backend.eval.calibrate_wasb --trajectory traj.csv --labels match.rallies.csv
--fps 59.94 --frame-width 1920
38 python -m backend.tools.rally_slicer --video match.mp4 --labels match.rallies.csv
--rallies all
39
40 # Example ? register as curated golden data in sports-data-collector:
41 python -m src.cli.curate import-local --sport badminton --video-path match.mp4
--annotations-path match.rallies.csv
42 ```
43
44 The `sport` column is self-documenting; tools that take a `--sport` flag can ignore it.
45

```

8. user (2026-06-21T13:35:02.827Z)

```

backend\worker\__init__.py
backend\worker\__main__.py

```

9. user (2026-06-21T13:35:02.840Z)

```

backend\eval\__init__.py
backend\eval\classifier.py
backend\eval\training.py
backend\eval\gemini_refine.py
backend\eval\audio\__init__.py
backend\eval\audio\e1_probe.py
backend\eval\export_wasb_segments.py
backend\eval\metrics.py
backend\eval\regression.py
backend\eval\annotations.py
backend\eval\calibrate_wasb.py
backend\eval\calibration.py
backend\eval\eval_partial_labels.py
backend\eval\rally_seq_proto.py
backend\eval\calibrate_local.py
backend\eval\gt_loader.py
backend\eval\harness.py
backend\eval\eval_stats.py
backend\eval\score_calibration.py
backend\eval\batch.py
backend\eval\fusion_golden.py

```

```

backend\eval\per_user_gate.py
backend\eval\per_user_eval.py
backend\eval\per_user_regression.py
backend\eval\served_gate_a.py
backend\eval\distill_local.py
backend\eval\promotion.py
backend\eval\serve_contrast.py
backend\eval\splits.py
backend\eval\segmentation_metrics.py
backend\eval\tolerance_metrics.py
backend\eval>windowing.py
backend\eval\ablation_pdf.py
backend\eval\ablation.py
backend\eval\experiment.py
backend\eval\fusion_audio.py
backend\eval\fusion_compare.py
backend\eval\golden_fixtures.py
backend\eval\golden_real_fixtures.py
backend\eval\rally_seg_eval.py

```

10. assistant (2026-06-21T13:35:05.416Z)

Now let me read the worker train CLI, the eval gate, and the gt_loader. Let me look at the key files.

11. user (2026-06-21T13:35:06.512Z)

```

1      """Worker dispatcher ? pull from storage ? run the pipeline ? push outputs back.
2
3      python -m backend.worker infer --video-uri <uri> --out-uri <prefix> \
4          [--segmenter wasb_hybrid] [--config config.json] [--set k=v ...] [--scratch DIR]
5      python -m backend.worker train ...          # Gen-0 train-from-bucket (follow-up)
6
7      The URI scheme selects the storage backend via ``backend.storage`` (a bare path /
``local://``
8      is an identity passthrough; ``s3://`` / ``gcs://`` is an object store), so existing
local runs
9      are byte-for-byte unchanged. Combine with ``RALLY_DETECTOR_IMPL=stub`` and
10     ``--set indexing.skip_ai_handoff=true`` for a **secret-free, GPU-free** run (no Gemini
key) ?
11     the path validated end-to-end on DigitalOcean (``docs/SETUP_NEW_MACHINE.md``).
12
13     This is a thin orchestrator: it composes the SAME building blocks the ``backend.main``
CLI uses
14     (validator registry ? segmenter ? report), never a forked pipeline.
15     """
16     from __future__ import annotations
17
18     import argparse
19     import csv
20     import json
21     import logging
22     import os
23     import sys
24     import tempfile
25     from typing import Any, List
26
27     from backend import storage
28     from backend.compute import ComputeJobSpec, get_compute_backend
29     from backend.config import (
30         StartupCheckError,
31         enforce_startup_checks,
32         load_config,
33         probe_cuda_available,
34     )

```

```

35     from backend.infrastructure.database import Database
36     from backend.pipeline.segmenters.base import segmenter_registry
37     from backend.pipeline.validators.base import registry as validator_registry
38     from backend.reporting import emit_indexer_report
39     from backend.results import Failure
40     from backend.utils.hashing import compute_video_id
41
42     logger = logging.getLogger("backend.worker")
43
44
45     def _coerce(v: str) -> Any:
46         """Coerce a ``--set`` string value to bool/int/float where it round-trips, else
47         str."""
48         low = v.lower()
49         if low in ("true", "false"):
50             return low == "true"
51         for cast in (int, float):
52             try:
53                 return cast(v)
54             except ValueError:
55                 pass
56         return v
57
58     def _apply_overrides(config: Any, sets: List[str]) -> None:
59         """Apply ``a.b.c=value`` overrides onto the typed config (e.g.
60         indexing.skip_ai_handoff=true)."""
61         for kv in sets or []:
62             key, sep, val = kv.partition("=")
63             if not sep:
64                 raise ValueError(f"--set expects k=v, got {kv!r}")
65             parts = key.split(".")
66             obj = config
67             for p in parts[:-1]:
68                 obj = getattr(obj, p)
69             setattr(obj, parts[-1], _coerce(val))
70
71     def _write_intervals_csv(segments: List[dict], path: str) -> None:
72         """Decimal-second ``[start,end)`` rallies + shot count / ending reason ? the join-
73         friendly
74         artifact (same schema as the rally-annotator labels)."""
75         with open(path, "w", newline="") as f:
76             w = csv.writer(f)
77             w.writerow(["start", "end", "shot_count", "ending_reason"])
78             for s in segments:
79                 w.writerow([
80                     f'{float(s.get("start_time", 0.0)):.3f}',
81                     f'{float(s.get("end_time", 0.0)):.3f}',
82                     s.get("shot_count", ""),
83                     s.get("ending_reason", s.get("shot_count_confidence", "")),
84                 ])
85
86     def _write_report_json(video_id: str, segments: List[dict], status: str, path: str) ->
87     None:
88         with open(path, "w") as f:
89             json.dump({
90                 "video_id": video_id,
91                 "status": status,
92                 "segment_count": len(segments),
93                 "segments": [{
94                     "start": float(s.get("start_time", 0.0)),
95                     "end": float(s.get("end_time", 0.0))
96                 } for s in segments],
97             }, f, indent=2)

```



```

96
97     def _run_startup_checks(config: Any) -> bool:
98         """M5 per-profile startup checks for the worker (the compute process, so it probes
CUDA
99         best-effort). Logs warnings; returns False on a FATAL coherence error so the caller
aborts
100         cleanly (rc 2) rather than run an expensive job under an incoherent profile. A true
no-op
101         (returns True, ZERO work) when no deployment.profile is selected.
102
103         The CUDA probe is gated behind an active profile: ``probe_cuda_available()`` lazily
imports
104         torch, and pre-M5 the worker's infer/train path was torch-free (detection is
delegated to a
105         WSL/native subprocess). Probing only when a profile is set keeps the default-OFF
path a true
106         no-op of work, not just of output."""
107         profile = getattr(getattr(config, "deployment", None), "profile", "") or ""
108         cuda = probe_cuda_available() if profile else None # torch-free on the default-OFF
path
109         try:
110             enforce_startup_checks(config, cuda_available=cuda, logger=logger)
111             return True
112         except StartupCheckError:
113             return False # already logged at ERROR by enforce_startup_checks
114
115
116     def _dispatch_remote(compute: Any, args: argparse.Namespace) -> int:
117         """M6: hand ONE infer job to a remote ComputeBackend (Cloud Run) and return its rc
once the
118         outputs are durably in the bucket (the backend bucket-polls the done-marker). The
dispatched
119         container runs this SAME cmd_infer INLINE (compute_target forced local_gpu) ? never
re-dispatch.
120
121         A remote job that never writes a completion marker (a failed/hung container) makes
the bucket
122         poll exhaust its budget ? ``PolicyTimeout``. Catch it and return a CLEAN rc 4
(remote dispatch
123         timeout/failure, distinct from the local 2/3) rather than letting a raw traceback
escape."""
124         from backend.infrastructure.execution_policy import PolicyTimeout
125
126         spec = ComputeJobSpec(
127             video_uri=args.video_uri, out_uri=args.out_uri,
128             segmenter=args.segmenter, config_overrides=tuple(args.set or ()),
129         )
130         try:
131             result = compute.run_infer(spec)
132         except PolicyTimeout as e:
133             logger.warning("[worker] remote compute did not complete (no marker within
budget): %s", e)
134             return 4
135             logger.info("[worker] remote compute done: video_id=%s rc=%d outputs=%s",
136                         result.video_id, result.returncode, result.outputs_uri)
137         return result.returncode
138
139
140     def _write_done_marker(done_uri: str, video_id: str, returncode: int, segment_count:
int,
141                           status: str, storage_cfg: dict, scratch: str) -> None:
142         """M6: write a tiny JSON completion marker to ``done_uri`` (via the storage layer)
for a remote
143         dispatcher's bucket-poll. Best-effort ? never raises into the worker's success
return."""

```

```

144     try:
145         marker = {"video_id": video_id, "returncode": returncode,
146                  "segment_count": segment_count, "status": status}
147         local = os.path.join(scratch, "_done.json")
148         with open(local, "w", encoding="utf-8") as f:
149             json.dump(marker, f)
150         storage.put(local, done_uri, config=storage_cfg)
151         logger.info("[worker] wrote completion marker -> %s", done_uri)
152     except Exception as e: # never fail a good run because the marker push hiccuped
153         logger.warning("[worker] could not write done-marker %s: %s", done_uri, e)
154
155
156     def cmd_infer(args: argparse.Namespace) -> int:
157         config = load_config(args.config) if args.config else load_config()
158         _apply_overrides(config, args.set)
159         if not _run_startup_checks(config):
160             return 2
161
162         # M6: a cloud compute_target DISPATCHES to a Cloud Run Job instead of running in-
163         # process.
164         # DEFAULT-OFF: get_compute_backend returns None for ""/local_gpu/cpu_mock ? fall
165         # through to the
166         # unchanged inline path below. The dispatched container runs THIS cmd_infer inline
167         # (the
168         # dispatcher forces compute_target=local_gpu), so it never recursively re-
169         # dispatches.
170         compute = get_compute_backend(config)
171         if compute is not None:
172             return _dispatch_remote(compute, args)
173
174         storage_cfg = config.storage.model_dump()
175
176         scratch = args.scratch or tempfile.mkdtemp(prefix="rally-worker-")
177         os.makedirs(scratch, exist_ok=True)
178         config.output.output_dir = scratch
179
180         # 1. PULL ? local:// / bare path = identity passthrough; s3:// / gcs:// = download
181         # to scratch.
182         local_video = storage.fetch(args.video_uri, os.path.join(scratch, "in.mp4"),
183                               config=storage_cfg)
184         print(f"[worker] pulled {args.video_uri} -> {local_video}")
185
186         # 2. IDENTITY ? whole-file MD5 (the labels?video join key; pulled bytes must be
187         # byte-identical).
188         video_id = compute_video_id(local_video)
189
190         # 3. VALIDATE (same registry as the main CLI).
191         val_results, val_context = validator_registry.run_all_with_context(local_video,
192                               config)
193
194         failures = [r for r in val_results if not r.passed]
195         db = Database(os.path.join(scratch, config.output.db_name))
196         db.add_video(video_id, local_video, "failed" if failures else "processed",
197                     [r.model_dump() for r in val_results])
198
199         segments: List[dict] = []
200         segmenter_actual = None
201         segmentation_ran = False
202         status = "failed"
203         try:
204             if failures:
205                 print("[worker] validation FAILED: "
206                       + "; ".join(f"{r.validator_name}: {r.message}" for r in failures))
207                 return 2
208             seg_name = args.segmenter or config.indexing.default_segmenter
209             segmenter_cls = segmenter_registry.get(seg_name)

```

```

201         if not segmenter_cls:
202             print(f"[worker] unknown segmenter: {seg_name!r} "
203                   f"(have {list(segmenter_registry.list_available())})")
204             return 2
205         segmenter_actual = seg_name
206         result = segmenter_cls(db, config).process_video(video_id, local_video,
max_frames=args.max_frames)
207         segmentation_ran = True
208         if isinstance(result, Failure):
209             print(f"[worker] segmenter FAILED: {result.message}")
210             return 3
211         segments = result.value if hasattr(result, "value") else result
212         status = "success"
213         # Loud, never-silent diagnostic: a 0-segment success is almost always a
misconfig, not an
214         # empty match ? the cold-start failure mode (the substrate detector yielded no
usable
215         # trajectories). Explain it so a run is analysable from the log alone (re-run -v
for more).
216         if not segments:
217             detector_impl = (os.environ.get("RALLY_DETECTOR_IMPL")
or getattr(config.indexing, "detector_impl", None) or
"auto")
219             logger.warning(
220                 "infer produced 0 segments for video_id=%s (segmenter=%s,
detector_impl=%s). "
221                 "The detector substrate yielded no usable trajectories/windows ? check
the detector "
222                 "logs above; common causes: native runner UNAVAILABLE
(weights/repo/WASB_PYTHON env), "
223                 "the WASB env produced no detections, or the input resolution differs
from the tuned "
224                 "proxy resolution. Re-run with -v for the native command + frame
counts.",
225                 video_id, segmenter_actual, detector_impl,
226             )
227         finally:
228             # Best-effort per-video observability report (never raises) ? parity with the
main CLI.
229             emit_indexer_report(
230                 db=db, video_id=video_id, video_path=local_video, config=config,
231                 val_results=val_results, val_context=val_context,
232                 segmenter_requested=args.segmenter, segmenter_actual=segmenter_actual,
233                 was_reingest=False, segmentation_ran=segmentation_ran,
234             )
235
236         # 5. SERIALIZE + 6. PUSH (outputs/<video_id>/?). idempotent on video_id.
237         out_local = os.path.join(scratch, "outputs", video_id)
238         os.makedirs(out_local, exist_ok=True)
239         _write_intervals_csv(segments, os.path.join(out_local, "intervals.csv"))
240         _write_report_json(video_id, segments, status, os.path.join(out_local,
"report.json"))
241
242         out_prefix = args.out_uri.rstrip("/")
243         pushed = []
244         for fn in sorted(os.listdir(out_local)):
245             uri = f"{out_prefix}/outputs/{video_id}/{fn}"
246             storage.put(os.path.join(out_local, fn), uri, config=storage_cfg)
247             pushed.append(uri)
248
249         print(f"[worker] infer done: {len(segments)} segment(s); pushed {len(pushed)}
artifact(s):")
250         for u in pushed:
251             print("    ", u)
252

```

```

253         # M6: a remote dispatcher passes --done-uri; write a completion marker so its
bucket-poll
254         # terminates (carries video_id ? the dispatcher's output path + the worker rc).
DEFAULT-OFF:
255         # no --done-uri ? no marker (unchanged local run). Written on SUCCESS only ? a
failed container
256         # job currently lets the dispatcher's bounded poll time out (a failure-marker is a
follow-up).
257         if getattr(args, "done_uri", None):
258             _write_done_marker(args.done_uri, video_id, 0, len(segments), status,
storage_cfg, scratch)
259         return 0
260
261
262         #: Video extensions accepted under <corpus>/videos/ (mirrors the labels/ '.csv' filter
so a
263         #: per-video sidecar ? .srt/.json/thumbnail ? can't shadow the real video on a stem
collision).
264         _VIDEO_EXTS = ("mp4", ".mov", ".mkv", ".avi", ".webm", ".m4v")
265
266
267     def _probe_video_fps_width(path: str):
268         """Robust ffprobe (fps, frame_width) for a real video. Returns None on ANY failure.
269
270         The detector trajectory is frame-indexed; the harness maps frame->time via fps, so a
WRONG
271         fps silently mis-aligns every rally label (e.g. a 60fps clip read as 30 doubles
every time).
272         cv2's ``_probe_fps_width`` silently defaults to (30, 1280) on a read miss ? fine for
the
273         inference report (it flags the fallback) but corrupting for REAL training data. So
here we
274         probe with ffprobe (same tool as ``get_video_duration``) and return None so the
caller SKIPS
275         rather than guesses.
276         """
277         import json
278         import subprocess
279         try:
280             out = subprocess.check_output(
281                 ["ffprobe", "-v", "error", "-select_streams", "v:0",
282                  "-show_entries", "stream=r_frame_rate,width", "-of", "json", path],
283                 stderr=subprocess.DEVNULL).decode()
284             st = (json.loads(out).get("streams") or [{}])[0]
285             num, _, den = str(st.get("r_frame_rate", "0/1")).partition("/")
286             fps = float(num) / float(den or "1")
287             width = float(st.get("width") or 0)
288             if fps > 0 and width > 0:
289                 return fps, width
290         except (subprocess.SubprocessError, ValueError, KeyError, OSError,
json.JSONDecodeError):
291             pass
292         return None
293
294
295     def _resolve_train_detector(args: argparse.Namespace, config: Any):
296         """Build the trajectory DetectorRunner for ``--trajectory-source detector``, honouring
297         detector_impl (RALLY_DETECTOR_IMPL / indexing.detector_impl: auto=WSL, native=GPU
box,
298         stub=CPU mock). Returns the runner, or prints an error + returns None.
299
300         Reuses the segmenter's ``_resolve_runner`` seam so the worker and the live CLI build
the
301         detector identically (one source of truth). A non-trajectory segmenter (e.g. yolo)
has

```

```

302     no shuttle detector and is rejected.
303     """
304     from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
305
306     seg_cls = segmenter_registry.get(args.segmenter)
307     if not seg_cls:
308         print(f"[worker] unknown segmenter: {args.segmenter!r} "
309               f"(have {list(segmenter_registry.list_available())})")
310         return None
311     seg = seg_cls(None, config)
312     if not isinstance(seg, TrajectoryHybridSegmenter):
313         print(f"[worker] --segmenter {args.segmenter!r} has no shuttle detector; "
314               f"use a trajectory hybrid (wasb_hybrid / tracknet_hybrid / "
315               f"fusion_hybrid).")
316         return None
317     try:
318         runner, _ = seg._resolve_runner(config.get("indexing", {}))
319     except NotImplementedError as e:
320         # e.g. detector_impl='native' for a family without a native runner (tracknet, or
321         # fusion with a tracknet substrate). Fail cleanly (rc!=0), not with a raw
322         traceback.print_exc()
323         print(f"[worker] detector not available: {e}")
324         return None
325     ok, msg = runner.healthcheck()
326     if not ok:
327         print(f"[worker] detector environment not ready: {msg}")
328         return None
329     print(f"[worker] detector ready ({args.segmenter}): {msg}")
330     return runner
331
332 def cmd_train(args: argparse.Namespace) -> int:
333     """Gen-0 train-from-bucket: pull labels (+ videos) -> trajectories -> manifest ->
334     shell out
335     to training.gen0.harness (backend must NOT import training) -> push the artifact
336     bundle.
337     Two trajectory sources (the train pipeline + harness are identical either way):
338     * ``synth`` (default) ? the CPU "mock GPU": deterministic, label-consistent
339     synthetic
340     shuttle paths (no GPU/WSL), so the whole pipeline is validated on a CPU box /
341     CI.
342     * ``detector`` ? REAL shuttle trajectories: pull each video and run the resolved
343     This
344     DetectorRunner (native WASB on a GPU box, or stub for a GPU-free wiring test).
345     is the M3.5 "first real training" path; only the trajectory source flips.
346     """
347     import subprocess
348
349     config = load_config(args.config) if args.config else load_config()
350     _apply_overrides(config, args.set)
351     if not _run_startup_checks(config):
352         return 2
353     storage_cfg = config.storage.model_dump()
354
355     scratch = args.scratch or tempfile.mkdtemp(prefix="rally-train-")
356     labels_dir = os.path.join(scratch, "labels")
357     traj_dir = os.path.join(scratch, "trajectories")
358     videos_dir = os.path.join(scratch, "videos")
359     os.makedirs(labels_dir, exist_ok=True)
360     os.makedirs(traj_dir, exist_ok=True)
361
362     corpus = args.corpus_uri.rstrip("/")
363     label_uris = sorted(u for u in storage.list_uris(f"{corpus}/labels/",
364 config=storage_cfg))

```

```

359                 if u.lower().endswith(".csv"))
360         if len(label_uris) < 2:
361             print(f"[worker] need >=2 labeled videos for leave-one-video-out; found
{len(label_uris)} "
362                   f"under {corpus}/labels/")
363             return 2
364
365         # Real-detector source: resolve the runner once + index the bucket's videos/ by
stem.
366         runner = None
367         video_by_stem: dict = {}
368         if args.trajectory_source == "detector":
369             os.makedirs(videos_dir, exist_ok=True)
370             runner = _resolve_train_detector(args, config)
371             if runner is None:
372                 return 2
373             for vuri in storage.list_uris(f"{corpus}/videos/", config=storage_cfg):
374                 vname = storage.parse_uri(vuri).name
375                 if not vname.lower().endswith(_VIDEO_EXTS):
376                     continue # skip sidecars (.srt/.json/thumbnails) so they can't shadow
the video
377                 video_by_stem[os.path.splitext(vname)[0]] = vuri
378
379             print(f"[worker] trajectory source: {args.trajectory_source}")
380             specs: List[dict] = []
381             for uri in label_uris:
382                 fname = storage.parse_uri(uri).name # e.g.
GX020094_proxy.rallies.csv
383                 name = fname.replace(".rallies.csv", "").replace(".csv", "")
384                 local_label = storage.fetch(uri, os.path.join(labels_dir, fname),
config=storage_cfg)
385                 if args.trajectory_source == "detector":
386                     vuri = video_by_stem.get(name)
387                     if not vuri:
388                         print(f"[worker] no video for {name!r} under {corpus}/videos/ ?
skipping.")
389                         continue
390                     local_video = storage.fetch(vuri, os.path.join(videos_dir,
storage.parse_uri(vuri).name),
391                                           config=storage_cfg)
392                     video_id = compute_video_id(local_video)
393                     # Real fps/width drive the trajectory frame->time mapping; PROBE (don't
guess) ? a
394                     # wrong fps silently mis-aligns every label, so skip the video if ffprobe
can't read it.
395                     meta = _probe_video_fps_width(local_video)
396                     if meta is None:
397                         print(f"[worker] could not probe fps/width for {name!r} (ffprobe) ?
skipping (won't guess).")
398                         continue
399                     fps, frame_width = meta
400                     traj = runner.run_predict(local_video, traj_dir, video_id=video_id)
401                     if not traj:
402                         print(f"[worker] detector produced no trajectory for {name!r} ?
skipping.")
403                         continue
404                     specs.append({"name": name, "trajectory": traj, "labels": local_label,
405                                "fps": fps, "frame_width": frame_width})
406             else:
407                 from backend.pipeline.detectors.stub_runner import
synth_trajectory_from_labels
408                 traj = os.path.join(traj_dir, f"{name}_traj.csv")
409                 synth_trajectory_from_labels(local_label, traj, fps=args.fps,
frame_width=args.frame_width)
410                 specs.append({"name": name, "trajectory": traj, "labels": local_label,

```

```

411         "fps": args.fps, "frame_width": args.frame_width})
412
413     if len(specs) < 2:
414         print(f"[worker] need >=2 videos with trajectories for leave-one-video-out; got
{len(specs)}.")
415         return 2
416     manifest_path = os.path.join(scratch, "manifest.json")
417     with open(manifest_path, "w", encoding="utf-8") as f:
418         json.dump(specs, f, indent=2)
419     src_label = "real detector" if args.trajectory_source == "detector" else "CPU-mock
stub"
420     print(f"[worker] built manifest: {len(specs)} videos ({src_label} trajectories)")
421
422     out_dir = os.path.join(scratch, "artifacts")
423     cmd = [sys.executable, "-m", "training.gen0.harness",
424            "--manifest", manifest_path, "--out", out_dir, "--version", args.version]
425     if args.floor:
426         floor_local = (storage.fetch(args.floor, os.path.join(scratch, "floor.json"),
config=storage_cfg)
427                        if "://" in args.floor else args.floor)
428         cmd += ["--floor", floor_local]
429     print("[worker] training:", " ".join(cmd))
430     rc = subprocess.run(cmd).returncode
431     if rc != 0:
432         print(f"[worker] gen0 harness failed (rc={rc})")
433         return rc
434
435     # PUSH the versioned artifact bundle (weights.json + manifest.json) to the bucket.
436     bundle = os.path.join(out_dir, args.version)
437     out_prefix = args.out_uri.rstrip("/")
438     pushed = []
439     for fn in sorted(os.listdir(bundle)):
440         uri = f"{out_prefix}/weights/{args.version}/{fn}"
441         storage.put(os.path.join(bundle, fn), uri, config=storage_cfg)
442         pushed.append(uri)
443     print(f"[worker] train done: pushed {len(pushed)} artifact(s):")
444     for u in pushed:
445         print("    ", u)
446     return 0
447
448
449     def build_parser() -> argparse.ArgumentParser:
450         p = argparse.ArgumentParser(prog="python -m backend.worker",
451                                     description="Storage-aware worker: pull ? run pipeline ?
push.")
452         p.add_argument("-v", "--verbose", action="store_true",
453                        help="DEBUG logging (the native detector command, frame counts, per-
stage detail)")
454         sub = p.add_subparsers(dest="cmd", required=True)
455
456         pi = sub.add_parser("infer", help="run inference on one video URI and push outputs")
457         pi.add_argument("--video-uri", required=True, help="local path / local:// / s3:// /
gcs://")
458         pi.add_argument("--out-uri", required=True, help="output prefix
(outputs/<video_id>/? is appended)")
459         pi.add_argument("--segmenter", default=None, help="default:
indexing.default_segmenter")
460         pi.add_argument("--config", default=None, help="config.json path (default:
./config.json)")
461         pi.add_argument("--scratch", default=None, help="local scratch dir (default: a temp
dir)")
462         pi.add_argument("--max-frames", type=int, default=None)
463         pi.add_argument("--done-uri", default=None,
464                        help="M6: storage URI for a completion MARKER (video_id + rc)
written on a "

```

```

465             "successful run, so a remote (Cloud Run) dispatcher's bucket-
poll terminates. "
466             "Default-OFF: omit it for the normal local run.")
467     pi.add_argument("--set", action="append", default=[], metavar="k=v",
468                     help="config override, e.g. --set indexing.skip_ai_handoff=true")
469     pi.set_defaults(func=cmd_infer)
470
471     pt = sub.add_parser("train", help="Gen-0 train-from-bucket (labels [+videos] ->
trajectories -> harness)")
472     pt.add_argument("--corpus-uri", required=True, help="prefix containing labels/ (+
videos/ for "
473                     "--trajectory-source detector), e.g. s3://bucket or gcs://bucket")
474     pt.add_argument("--out-uri", required=True, help="output prefix (weights/<version>/
is appended)")
475     pt.add_argument("--version", required=True, help="semver for the artifact bundle")
476     pt.add_argument("--trajectory-source", choices=["synth", "detector"],
default="synth",
477                     help="synth = CPU-mock label-consistent trajectories (default, no
GPU); "
478                     "detector = REAL shuttle trajectories via the resolved
DetectorRunner "
479                     "(needs videos/ in the corpus; honours detector_impl).")
480     pt.add_argument("--segmenter", default="wasb_hybrid",
481                     help="trajectory hybrid whose detector is used for --trajectory-
source detector")
482     pt.add_argument("--floor", default=None, help="floor baseline JSON (local path or
storage URI)")
483     pt.add_argument("--config", default=None)
484     pt.add_argument("--scratch", default=None)
485     pt.add_argument("--fps", type=float, default=30.0, help="synth trajectory fps
(detector probes the video)")
486     pt.add_argument("--frame-width", type=float, default=1280.0,
487                     help="synth trajectory frame width (detector probes the video)")
488     pt.add_argument("--set", action="append", default=[], metavar="k=v")
489     pt.set_defaults(func=cmd_train)
490     return p
491
492
493     def _setup_logging(verbose: bool) -> None:
494         """Configure logging for the worker CLI so the pipeline's existing INFO/ERROR
diagnostics are
495         VISIBLE ? the worker previously ran with no logging config, so every
segmenter/detector log
496         (detector choice, 'Running native WASB inference', availability failures) was
silently dropped.
497         That is exactly why a 0-rally cold-start gave nothing to analyse. Configured here
(the CLI
498         entry), not at import, so importing the module never reconfigures a caller's
logging."""
499         # force=True: basicConfig is a no-op if the root already has handlers (e.g. a
library
500         # configured logging at import), which would leave the level at WARNING and re-
suppress the
501         # diagnostics. Forcing reconfiguration means the worker reliably controls its own
verbosity.
502         logging.basicConfig(
503             level=logging.DEBUG if verbose else logging.INFO,
504             format="%(asctime)s [% (levelname)s] %(name)s: %(message)s",
505             datefmt="%H:%M:%S",
506             force=True,
507         )
508
509
510     def main(argv=None) -> int:
511         args = build_parser().parse_args(argv)

```



```

512     _setup_logging(getattr(args, "verbose", False))
513     return args.func(args)
514
515
516 if __name__ == "__main__":
517     sys.exit(main())
518

```

12. user (2026-06-21T13:35:06.553Z)

```

1     """Canonical ground-truth interval loader ? THE one GT/golden-CSV reader (ADR-008
prereq).
2
3     The repo historically carried two incompatible loaders:
4     - ``annotations.load_annotations``: positional ``start,end`` (the collector's
5       ``curate export`` format), raises on bad rows;
6     - ``calibrate_wasb.load_golden_gts``: DictReader requiring ``start_time/end_time``
7       (the rally-annotator golden format), silently skipping bad rows ? fed 7 modules.
8
9     A file in one convention fed to the other loader yielded zero/garbage intervals with no
10    error. This module accepts BOTH conventions (header-mapped, else positional), so every
11    quantity the ship-gate depends on (gt_hash, LOVO scores, the future consent fingerprint)
12    flows through one reader. Both legacy entry points now delegate here.
13
14    Accepted shapes (one rally per row; blank lines and ``#`` comments ignored):
15    - header with ``start``/``end`` OR ``start_time``/``end_time`` columns (any other
16      columns ? rally_number, ending_reason, sport, ? ? are ignored);
17    - no header: columns 0,1 positionally.
18    Timestamps: decimal seconds or ``MM:SS`` / ``HH:MM:SS``
19    (``annotations.parse_timestamp``).
20    """
21    import csv
22    import logging
23    from typing import List, Optional, Tuple
24
25    from backend.eval.annotations import parse_timestamp
26
27    logger = logging.getLogger(__name__)
28
29    Interval = Tuple[float, float]
30
31    _START_NAMES = ("start", "start_time")
32    _END_NAMES = ("end", "end_time")
33
34    def _header_columns(row: List[str]) -> Optional[Tuple[int, int]]:
35        """If `row` is a header naming start/end columns, return their indices, else
36        None."""
37        cells = [(c or "").strip().lower() for c in row]
38        start_idx = next((i for i, c in enumerate(cells) if c in _START_NAMES), None)
39        end_idx = next((i for i, c in enumerate(cells) if c in _END_NAMES), None)
40        if start_idx is not None and end_idx is not None:
41            return start_idx, end_idx
42        return None
43
44    def load_gt_intervals(csv_path: str, *, strict: bool = True) -> List[Interval]:
45        """Load GT rally intervals from either golden-CSV convention.
46
47        strict=True (corpus/gate use): raise ValueError with file:line on any malformed row
48        (bad timestamp, end <= start, negative start) so labeling mistakes fail loudly.
49        strict=False (legacy tuning-driver behavior): skip bad rows, but LOG how many were
50        skipped ? a silently shrinking GT set flatters precision.
51
52        Returns intervals sorted by start; overlapping GT intervals are logged as a warning

```

```

53         (they break one-to-one matching and usually indicate a labeling error).
54         """
55         intervals: List[Interval] = []
56         skipped = 0
57         cols: Optional[Tuple[int, int]] = None    # (start_idx, end_idx); None until detected
58         saw_header = False
59
60         with open(csv_path, newline="", encoding="utf-8-sig") as f:
61             for lineno, row in enumerate(csv.reader(f), start=1):
62                 if not row or not (row[0] or "").strip() or row[0].strip().startswith("#"):
63                     continue
64                 if cols is None and not saw_header:
65                     header = _header_columns(row)
66                     if header is not None:
67                         cols = header
68                         saw_header = True
69                     continue    # consume the header row
70                 cols = (0, 1)    # headerless: positional start,end
71                 assert cols is not None    # every branch above assigns it
72                 si, ei = cols
73                 try:
74                     if len(row) <= max(si, ei):
75                         raise ValueError(f"expected at least {max(si, ei) + 1} columns, got
{row!r}")
76                     start = parse_timestamp(row[si])
77                     end = parse_timestamp(row[ei])
78                     if start < 0:
79                         raise ValueError(f"negative start ({start})")
80                     if start >= end:
81                         raise ValueError(f"start ({start}) >= end ({end})")
82                 except ValueError as e:
83                     if strict:
84                         raise ValueError(f"{csv_path}:{lineno}: {e}") from e
85                     skipped += 1
86                     continue
87                 intervals.append((start, end))
88
89         if skipped:
90             logger.warning("%s: skipped %d malformed GT row(s) (strict=False)", csv_path,
skipped)
91
92         intervals.sort(key=lambda iv: iv[0])
93         for (s1, e1), (s2, e2) in zip(intervals, intervals[1:]):
94             if s2 < e1:
95                 logger.warning("%s: overlapping GT intervals (%.3f-%.3f vs %.3f-%.3f) ? "
96                               "labeling error? Overlaps break one-to-one matching.",
97                               csv_path, s1, e1, s2, e2)
98         return intervals
99

```

13. assistant (2026-06-21T13:35:09.816Z)

Now let me look at the ship-gate (the licensing gate that refuses paid-LLM labels) and the training harness. Let me find the gate verdict and label_source logic.

14. user (2026-06-21T13:35:10.967Z)

```

Found 4 files
training\gen0\harness.py
tests\test_training_gen0.py
docs\archives\past_projects\DECOUPLED_COMPUTE\DEPLOY_REPORT_GCP_2026-06-20.md
artifacts\rally_tal_gen0\0.1.0\manifest.json

```

15. user (2026-06-21T13:35:11.279Z)

```
training\gen0\__init__.py
training\__init__.py
training\gen0\harness.py
```

16. user (2026-06-21T13:35:14.624Z)

```
1      """Gen-0 TAL harness (M0.4): ONE command -> train -> honest LOVO eval -> ship-gate ->
artifact.
2
3      python -m training.gen0.harness --manifest output/human_lovo_manifest.json \
4      --floor eval_baselines/heuristic_lovo_n6.json --out artifacts/rally_tal_gen0
--version 0.1.0
5
6      Productionizes the rally_seq prototype per ADR-008:
7      - TRAINS on the corpus manifest (per-video trajectory + golden labels), evaluates with
8      honest leave-one-video-out (split by match, threshold tuned on train only), then fits
9      the FINAL model on all videos.
10     - Emits a versioned ARTIFACT BUNDLE ? the only thing serving may ever consume:
11         <out>/<version>/weights.json      (gitignored; classifier + frozen normalization)
12         <out>/<version>/manifest.json      (the reviewable contract: feature-schema version,
13                                           calibration, training lineage + GT hashes,
14                                           eval results, attestations, gate verdict)
15     - Runs the SHIP-GATE as pre-flight: floor comparison + a PAIRED per-video sign test
16       (at n=6 a mean gap alone is noise-ambiguous), plus the hard attestation blocks
17       (WASB C3 unresolved => commercial ship refused; explicit F1@tIoU target still
18       undefined => no self-declared victory). The product side owns final admission
19       (backend/eval/regression.py); this harness never grades itself into serving.
20
21     Training may import backend.* (one-way wall); backend must never import training.*.
22     """
23     from __future__ import annotations
24
25     import argparse
26     import hashlib
27     import json
28     import os
29     import re
30     import subprocess
31     import time
32     from math import comb
33     from typing import Dict, List, Optional
34
35     import numpy as np
36
37     from backend.eval.classifier import LogisticRegression
38     from backend.eval.gt_loader import load_gt_intervals
39     from backend.eval.regression import gt_hash
40     from backend.eval.rally_seq_proto import (
41         labels_for, run as lovo_run, _seg_f1,
42     )
43     from backend.features.rally_seq import FEAT_NAMES, FEATURE_SCHEMA, build_frame_arrays,
features
44     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
45
46     # Post-processing constants ? part of the model's identity, recorded in the manifest.
47     CALIBRATION_DEFAULTS = {"smooth": 7, "merge_gap": 1.0, "min_dur": 1.0}
48     THRESHOLD_GRID = [round(t, 2) for t in np.arange(0.2, 0.81, 0.05)]
49
50
51     # Repo root = two levels up from training/gen0/harness.py. A `GIT_SHA` file baked here
at
52     # tarball-build time is the off-box provenance fallback (see _git_sha +
deploy/gcp/README.md).
53     _REPO_ROOT = os.path.abspath(os.path.join(os.path.dirname(__file__), "..", ".."))
54     _SHA_RE = re.compile(r"^[0-9a-f]{7,40}$")
```

```

55
56     #: Env var carrying the source commit SHA out-of-band when the run has no `.git` (a `git
archive`
57     #: tarball ? the GCP-L4 path). Set it on the box, or bake a `GIT_SHA` file into the
archive root.
58     GIT_SHA_ENV = "RALLY_GIT_SHA"
59
60
61     def _valid_sha(s: Optional[str]) -> Optional[str]:
62         """Return the normalized commit SHA if `s` looks like one (7-40 hex chars), else
None.
63
64         Guards every provenance source so a stray value (`unknown\n`, a build label) can
never be
65         stamped as if it were a real SHA."""
66         s = (s or "").strip().lower()
67         return s if _SHA_RE.match(s) else None
68
69
70     def _git_sha() -> str:
71         """Resolve the source commit SHA for the manifest, in priority order:
72
73         1. live `git rev-parse HEAD` ? a real checkout (local dev / CI); authoritative,
unchanged.
74         2. the `RALLY_GIT_SHA` env var ? out-of-band injection for a run off a `git
archive`
75         tarball, which carries no `.git` (the 2026-06-20 GCP-L4 box stamped `unknown`
here).
76         3. a `GIT_SHA` file baked into the repo root at tarball-build time.
77         4. `unknown` ? nothing available (last resort; never fabricated).
78
79         Steps 2-4 make a remote tarball run provenance-stamped WITHOUT changing the git-
present path.
80         """
81         try:
82             out = subprocess.run(["git", "rev-parse", "HEAD"], capture_output=True,
83                                 text=True, timeout=10).stdout
84             live = _valid_sha(out)
85             if live:
86                 return live
87         except Exception:
88             pass
89         env_sha = _valid_sha(os.environ.get(GIT_SHA_ENV))
90         if env_sha:
91             return env_sha
92         try:
93             with open(os.path.join(_REPO_ROOT, "GIT_SHA"), encoding="utf-8") as f:
94                 file_sha = _valid_sha(f.read())
95             if file_sha:
96                 return file_sha
97         except OSError:
98             pass
99         return "unknown"
100
101
102     def _sha256(path: str) -> str:
103         h = hashlib.sha256()
104         with open(path, "rb") as f:
105             for chunk in iter(lambda: f.read(65536), b''):
106                 h.update(chunk)
107         return h.hexdigest()
108
109
110     def sign_test(learned: List[float], floor: List[float]):
111         """Paired one-sided sign test: is learned > floor more often than chance?

```

```

112
113     At n=6 a mean comparison alone is inside noise (SE ~0.11); the paired per-video
114     wins are the defensible statistic. Returns (wins, n, p_one_sided)."""
115     pairs = [(lv, fv) for lv, fv in zip(learned, floor) if lv != fv] # ties drop,
standard
116     wins = sum(1 for lv, fv in pairs if lv > fv)
117     n = len(pairs)
118     p = sum(comb(n, k) for k in range(wins, n + 1)) / (2 ** n) if n else 1.0
119     return wins, n, p
120
121
122     def train_final(specs: List[Dict]):
123         """Fit the FINAL model on ALL corpus videos; threshold tuned on the same (train-
side).
124
125         The honest generalization estimate is the LOVO result, not this fit ? recorded as
such."""
126         vids = []
127         for s in specs:
128             pts = TrackNetRunner.parse_trajectory_csv(s["trajectory"])
129             arr = build_frame_arrays(pts, float(s["frame_width"]), float(s["fps"]))
130             X, times = features(arr, float(s["fps"]))
131             gts = load_gt_intervals(s["labels"], strict=False)
132             vids.append({"name": s["name"], "X": X, "times": times,
133                         "y": labels_for(times, gts), "gts": gts})
134         Xall = np.vstack([v["X"] for v in vids])
135         yall = np.concatenate([v["y"] for v in vids])
136         clf = LogisticRegression(lr=0.2, epochs=2000, l2=1e-2)
137         clf.fit(Xall, yall, feature_names=FEAT_NAMES)
138         thr = max(THRESHOLD_GRID, key=lambda t: _seg_f1(clf, vids, float(t)))
139         return clf, float(thr)
140
141
142     def gate_verdict(eval_res: dict, floor: Optional[dict]) -> dict:
143         """Pre-flight ship-gate.
144
145         Provisional F1 target (NOT owner-ratified; calibration tracked in #172).
146         Floor (necessary): heuristic LOVO ~0.480 (from quality table).
147         Multi-court ref (makes owned \"worth serving\" per data): 0.66.
148         Proposed for \"very high P/R\" + significance: F1@tIoU0.5 >= 0.70 on
149         multi-court folds with paired-sign p<0.05 vs heuristic (refine as corpus grows).
150         Beating floor + sign test necessary; C3 main commercial blocker.
151         ship=True only when all non-C3 blocks clear (honesty first).
152         """
153         reasons: List[str] = []
154         floor_passed = None
155         sign = None
156         if floor:
157             floor_mean = float(floor["mean_f1"])
158             floor_passed = eval_res["mean_f1"] > floor_mean
159             if not floor_passed:
160                 reasons.append(f"LOVO mean {eval_res['mean_f1']:.3f} <= floor
{floor_mean:.3f}")
161             per = floor.get("per_video", {})
162             paired = [(r["f1"], per[r["video"]]) for r in eval_res["per_video"] if
r["video"] in per]
163             if paired:
164                 wins, n, p = sign_test([lv for lv, _ in paired], [fv for _, fv in paired])
165                 sign = {"wins": wins, "n": n, "p_one_sided": round(p, 4)}
166                 if p >= 0.05:
167                     reasons.append(f"paired sign test not significant ({wins}/{n} wins,
p={p:.3f}) "
168                                     f"? grow the corpus before claiming superiority")
169             else:
170                 reasons.append("no floor baseline provided ? nothing to beat")

```

```

171
172     # Provisional blocker (per owner review on #170; see #172 for calibration).
173     # (The number itself is not the blocker; calibration tracked in #172.)
174     reasons.append("ship-gate F1@tIoU target NOT YET CALIBRATED ? provisional; see issue
#172")
175     reasons.append("WASB checkpoint provenance C3 UNRESOLVED ? commercial ship blocked
regardless of F1")
176     # ship remains False until C3 resolved + other gates (by design).
177     return {"ship": False, "floor_passed": floor_passed, "sign_test": sign, "blockers":
reasons}
178
179
180 def run(manifest_path: str, out_dir: str, version: str,
181         floor_path: Optional[str] = None) -> dict:
182     with open(manifest_path, encoding="utf-8") as f:
183         specs = json.load(f)
184         floor = None
185         if floor_path:
186             with open(floor_path, encoding="utf-8") as f:
187                 floor = json.load(f)
188
189         # 1) Honest LOVO eval (the number that matters).
190         eval_res = lovo_run(specs)
191
192         # 2) Final model on all videos.
193         clf, thr = train_final(specs)
194
195         # 3) Gate (pre-flight).
196         verdict = gate_verdict(eval_res, floor)
197
198         # 4) Artifact bundle.
199         bundle_dir = os.path.join(out_dir, version)
200         os.makedirs(bundle_dir, exist_ok=True)
201         weights_path = os.path.join(bundle_dir, "weights.json")
202         payload = clf.to_dict()
203         payload["calibration"] = {"threshold": thr, **CALIBRATION_DEFAULTS}
204         payload["feature_schema_version"] = FEATURE_SCHEMA["version"]
205         with open(weights_path, "w", encoding="utf-8") as f:
206             json.dump(payload, f, indent=2)
207
208         manifest = {
209             "artifact": {
210                 "name": "rally_tal_gen0", "version": version,
211                 "created_at": time.strftime("%Y-%m-%dT%H:%M:%S%z"),
212                 "git_sha": _git_sha(),
213                 "weights_file": "weights.json", "weights_sha256": _sha256(weights_path),
214             },
215             "feature_schema": FEATURE_SCHEMA,
216             "calibration": {"threshold": thr, **CALIBRATION_DEFAULTS,
217                             "note": "threshold tuned train-side on the full corpus; "
218                                     "honest generalization = the LOVO eval below"},
219             "training_lineage": {
220                 "corpus_manifest": manifest_path,
221                 "split_protocol": "leave-one-video-out by match",
222                 "label_source": "human (rally-annotator golden CSVs)",
223                 "videos": [{ "name": s["name"], "labels": s["labels"],
224                             "gt_hash": gt_hash(load_gt_intervals(s["labels"],
strict=False)),
225                             "fps": s["fps"]} for s in specs],
226             },
227             "eval": {
228                 "protocol": "LOVO, F1@tIoU=0.5 segment-level + per-frame AUC",
229                 "per_video": eval_res["per_video"],
230                 "mean_f1": eval_res["mean_f1"],
231                 "floor": floor,

```

```

232         },
233         "attestation": {
234             "deps_licenses": "numpy/scipy (BSD) + repo-internal code only (Gen-0)",
235             "wasb_checkpoint_provenance": "C3-UNRESOLVED (ship-blocking; see
COMMERCIALIZATION C3)",
236             "no_paid_llm_training_labels": True,
237             "consent_ledger": "pending-M0.1 (owner-attested: owned footage, all-adult
sessions)",
238         },
239         "gate": verdict,
240     }
241     manifest_out = os.path.join(bundle_dir, "manifest.json")
242     with open(manifest_out, "w", encoding="utf-8") as f:
243         json.dump(manifest, f, indent=2)
244
245     print(f"\n=== Gen-0 artifact bundle: {bundle_dir} ===")
246     print(f"weights : {weights_path}")
247     print(f"manifest: {manifest_out}")
248     print(f"GATE      : ship={verdict['ship']} floor_passed={verdict['floor_passed']} "
249           f"sign_test={verdict['sign_test']}")
250     for b in verdict["blockers"]:
251         print(f" - {b}")
252     return manifest
253
254
255     def main() -> None:
256         ap = argparse.ArgumentParser(description=__doc__)
257         ap.add_argument("--manifest", required=True,
258                         help="corpus manifest JSON:
[{'name','trajectory','fps','frame_width','labels'}, ...]")
259         ap.add_argument("--floor", default=None,
260                         help="floor baseline JSON (e.g.
eval_baselines/heuristic_lovo_n6.json)")
261         ap.add_argument("--out", default=os.path.join("artifacts", "rally_tal_gen0"))
262         ap.add_argument("--version", required=True, help="semver for the artifact bundle")
263         args = ap.parse_args()
264         run(args.manifest, args.out, args.version, floor_path=args.floor)
265
266
267     if __name__ == "__main__":
268         main()
269

```

17. assistant (2026-06-21T13:35:18.283Z)

Now let me look at the stub_runner (synth_trajectory_from_labels) and the eval gates referenced in the task (gate_verdict in regression.py, per_user_gate, served_gate_a). Let me also check for the ANNOTATE_AND_TRAIN doc in khelsutra and any in-browser annotation.

18. user (2026-06-21T13:35:21.341Z)

```

1     """CPU stub shuttle-trajectory runner ? "mock a GPU with a CPU".
2
3     A ``DetectorRunner`` that produces a deterministic ``Frame,Visibility,X,Y`` trajectory
4     CSV with no GPU, no WSL, no model weights ? so the whole trajectory-hybrid pipeline
5     (``wasb_hybrid`` / ``tracknet_hybrid``) can run end-to-end on a CPU-only box (CI, a CPU
6     container, a laptop) for regression testing. The person/yolo path already CPU-falls-back
7     (``use_gpu=False``); this closes the last GPU/WSL-only gap so the entire pipeline is
8     runnable ? and regression-gateable ? without a GPU.
9
10    Two modes:
11    * replay ? emit an existing ``Frame,Visibility,X,Y`` CSV verbatim (drive the
12      pipeline with a committed/golden trajectory; deterministic + cross-OS).
13    * synth (default) ? generate a deterministic synthetic trajectory of
14      ``num_rallies`` in-flight bursts (visible + fast-moving frames) separated by dead

```

```

15         gaps, so windowing yields a stable, known number of candidate windows.
16
17     This is the CPU seam the ``DetectorRunner`` ABC was built for (see ``base.py`` ?
18     "Optional Linux-native path") and the **seed of the future ``LinuxNativeRunner``**
19     (M3.5, ``docs/DECOUPLED_COMPUTE/04-BUILD-AND-TEST-PLAN.md``). It is selected via
20     ``indexing.detector_impl="stub"`` or the ``RALLY_DETECTOR_IMPL=stub`` env var; the real
21     WSL runners stay the default, so production behaviour is unchanged.
22     """
23     import os
24     import csv as _csv
25     import shutil
26     import logging
27     from dataclasses import dataclass
28     from typing import Any, Dict, List, Optional, Tuple
29
30     from backend.pipeline.detectors.base import DetectorRunner
31     # Reuse the model-agnostic CSV parsing + windowing (no torch/tf pulled here).
32     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
33
34     logger = logging.getLogger(__name__)
35
36
37     @dataclass
38     class StubConfig:
39         """Settings for the CPU stub runner (built from ``config.json`` ->
40         ``indexing.stub``)."""
41         fps: float = 30.0
42         frame_width: float = 1280.0
43         num_rallies: int = 2
44         rally_seconds: float = 4.0
45         gap_seconds: float = 3.0          # dead gap between rallies (> merge_gap so
46         windows don't merge)
47         lead_seconds: float = 2.0        # dead lead-in
48         step_px: float = 12.0            # per-frame shuttle displacement during a rally
49         (>> velocity_thresh)
50         replay_csv: Optional[str] = None # if set + exists, emit this CSV verbatim instead
51         of synthesising
52
53     @classmethod
54     def from_indexing_cfg(cls, idx_cfg: Dict[str, Any]) -> "StubConfig":
55         s = (idx_cfg or {}).get("stub", {}) or {}
56         return cls(
57             fps=float(s.get("fps", 30.0)),
58             frame_width=float(s.get("frame_width", 1280.0)),
59             num_rallies=int(s.get("num_rallies", 2)),
60             rally_seconds=float(s.get("rally_seconds", 4.0)),
61             gap_seconds=float(s.get("gap_seconds", 3.0)),
62             lead_seconds=float(s.get("lead_seconds", 2.0)),
63             step_px=float(s.get("step_px", 12.0)),
64             replay_csv=s.get("replay_csv"),
65         )
66
67     class StubDetectorRunner(DetectorRunner):
68         """Deterministic CPU trajectory runner (no GPU/WSL). See the module docstring."""
69
70         def __init__(self, cfg: Optional[StubConfig] = None):
71             self.cfg = cfg or StubConfig()
72
73         def healthcheck(self) -> Tuple[bool, str]:
74             return True, "stub CPU detector (no GPU/WSL)"
75
76         def _rows(self) -> List[Tuple[int, int, float, float]]:
77             """Deterministic ``(frame, visible, x, y)`` rows: ``num_rallies`` in-flight
78             bursts

```



```

75         separated by dead gaps. During a rally the shuttle oscillates ``step_px``/frame
76         ?
77         well above the default 0.02 velocity threshold ? so every rally frame is
78         "active"
79         and each burst becomes exactly one candidate window."""
80         cfg = self.cfg
81         rows: List[Tuple[int, int, float, float]] = []
82         frame = 0
83         y = cfg.frame_width / 4.0
84         base_x = cfg.frame_width / 2.0
85         for _ in range(int(cfg.lead_seconds * cfg.fps)):
86             rows.append((frame, 0, 0.0, 0.0))
87             frame += 1
88         for r in range(cfg.num_rallies):
89             for i in range(int(cfg.rally_seconds * cfg.fps)):
90                 x = base_x + (cfg.step_px if i % 2 else -cfg.step_px)
91                 rows.append((frame, 1, x, y))
92                 frame += 1
93             if r != cfg.num_rallies - 1:
94                 for _ in range(int(cfg.gap_seconds * cfg.fps)):
95                     rows.append((frame, 0, 0.0, 0.0))
96                     frame += 1
97         return rows
98
99     def run_predict(self, video_win_path: str, output_win_dir: str,
100                    video_id: Optional[str] = None) -> Optional[str]:
101         """Write a deterministic trajectory CSV and return its path (never touches the
102         GPU/WSL)."""
103         os.makedirs(output_win_dir, exist_ok=True)
104         stem = os.path.splitext(os.path.basename(str(video_win_path).replace("\\",
105         "/"")))[0]
106         key = f"{stem}__{video_id[:12]}" if video_id else (stem or "stub")
107         out_csv = os.path.join(output_win_dir, f"{key}_stub.csv")
108
109         if self.cfg.replay_csv and os.path.exists(self.cfg.replay_csv):
110             shutil.copyfile(self.cfg.replay_csv, out_csv)
111             logger.info("Stub runner: replayed trajectory %s -> %s",
112             self.cfg.replay_csv, out_csv)
113         return out_csv
114
115         rows = self._rows()
116         with open(out_csv, "w", newline="") as f:
117             w = _csv.writer(f)
118             w.writerow(["Frame", "Visibility", "X", "Y"])
119             for frame, vis, x, y in rows:
120                 w.writerow([frame, vis, f"{x:.3f}", f"{y:.3f}"])
121             logger.info("Stub runner: wrote %d synthetic trajectory rows -> %s", len(rows),
122             out_csv)
123         return out_csv
124
125         # Model-agnostic CSV parsing + windowing ? identical to the real runners.
126         parse_trajectory_csv = staticmethod(TrackNetRunner.parse_trajectory_csv)
127         trajectory_to_action_windows =
128         staticmethod(TrackNetRunner.trajectory_to_action_windows)
129
130     def synth_trajectory_from_labels(labels_csv: str, out_csv: str, *, fps: float = 30.0,
131                                    frame_width: float = 1280.0, tail_pad: float = 2.0,
132                                    step_px: float = 12.0) -> str:
133         """CPU "mock GPU" for TRAINING: write a deterministic Frame,Visibility,X,Y
134         trajectory whose
135         in-flight (visible + oscillating) frames cover each labeled rally window and whose
136         dead frames
137         cover the gaps ? so the synthetic trajectory *encodes* the rally labels.

```

```

131         This lets the Gen-0 harness train + leave-one-video-out eval end-to-end on coherent
data with
132         no GPU/WSL, validating the whole training pipeline before the real WASB detector
lands. (The
133         learned model is meaningful for plumbing, not for production ? the trajectory is
synthetic.)
134         """
135         rallies: List[Tuple[float, float]] = []
136         with open(labels_csv, newline="", encoding="utf-8") as f:
137             for row in _csv.DictReader(f):
138                 try:
139                     rallies.append((float(row["start_time"]), float(row["end_time"])))
140                 except (KeyError, ValueError):
141                     continue
142         duration = (max(e for _, e in rallies) + tail_pad) if rallies else 10.0
143         nframes = max(1, int(duration * fps))
144         base_x = frame_width / 2.0
145         y = frame_width / 4.0
146         os.makedirs(os.path.dirname(out_csv) or ".", exist_ok=True)
147         with open(out_csv, "w", newline="", encoding="utf-8") as f:
148             w = _csv.writer(f)
149             w.writerow(["Frame", "Visibility", "X", "Y"])
150             for i in range(nframes):
151                 t = i / fps
152                 if any(s <= t < e for s, e in rallies):
153                     x = base_x + (step_px if i % 2 else -step_px)
154                     w.writerow([i, 1, f"{x:.3f}", f"{y:.3f}"])
155                 else:
156                     w.writerow([i, 0, "0.000", "0.000"])
157         return out_csv
158

```

19. user (2026-06-21T13:35:21.345Z)

```

C:\Users\avidu\Projects\khelsutra\docs\REQUEST_ACCESS_FORM.md
C:\Users\avidu\Projects\khelsutra\docs\PORTABILITY.md
C:\Users\avidu\Projects\khelsutra\docs\ANNOTATE_AND_TRAIN.md
C:\Users\avidu\Projects\khelsutra\docs\PER_RALLY_SELECTION.md
C:\Users\avidu\Projects\khelsutra\docs\FRIENDLY_AND_MULTILINGUAL.md
C:\Users\avidu\Projects\khelsutra\docs\LOCAL_APPLIANCE_ARCHITECTURE.md
C:\Users\avidu\Projects\khelsutra\docs\README.md
C:\Users\avidu\Projects\khelsutra\docs\DRY_RUN_CUJ.md

```

20. user (2026-06-21T13:35:21.386Z)

```

C:\Users\avidu\Projects\khelsutra\docs\ANNOTATE_AND_TRAIN.md

```

21. assistant (2026-06-21T13:35:25.310Z)

Now let me read the khelsutra ANNOTATE_AND_TRAIN doc (this is the live design for the in-app workflow), plus the per_user_gate and the served_gate / regression gate to understand the full ship-gate story.

22. user (2026-06-21T13:35:25.639Z)

```

1         # Annotate your footage ? train your own model (KhelSutra Studio USP pages)
2
3         > **Status:** `DRAFT` ? owner-gated on scope . **Owner:** Avi . **Created:** 2026-06-20
. **Last updated:** 2026-06-20
4         > **Lifecycle:** `DRAFT` ? IN PROGRESS ? DONE ? archived` (move to `docs/archives/` when
DONE)
5         > **Tracking anchors:** $7 progress tracker is the source of truth; index in
`docs/README.md`; pointer in `NEXT_STEPS.md` + memory `current-state` once work starts.
6         > **Relation:** peer of `khelsutra/docs/PER_RALLY_SELECTION.md` and

```

```

`khelsutra/docs/LOCAL_APPLIANCE_ARCHITECTURE.md` (the latter lands with PR #13); mirrors the
engine repo's `PROJECT_DOC_TEMPLATE` discipline. Documents an EXTERNAL public tool (`rally-
annotator`, MIT) and the engine's `backend/worker train` CLI; it does not own those repos.
7   > Honesty note: every engine-fact claim is tagged `[verified]` (read against code
this session), `[design]` (proposed here), or `[researched]` (needs sign-off). Not legal advice.
8
9   ---
10
11   ## 0. TL;DR
12   Ship ONE new static marketing page (`site/public/own-model.html`, slug `/own-model`)
with three anchored sections ? Annotate your footage · Train your own model · Setup & use ?
telling the bring-your-own-model story and linking the real, public, MIT rally-annotator VLC
plugin `[verified]`. The single most important caveat: only the ANNOTATE half is real today
? the rally-annotator ships (v1.6.4, MIT) and its CSV is directly ingestible by the engine with
no transform `[verified]` ? while the TRAIN half is an open, in-build CLI whose GPU-free
path trains a pipeline-validation mock only, and whose real model needs a GPU + an in-
flight native WASB runner behind an owner-gated ship decision `[verified gate_verdict
harness.py:98]`. The Studio SPA gets a design-tracked, owner-gated correction/contribute
screen, never a "Train now" button. Defensible claim = ownership / licensing / privacy /
consent / minors-excluded, NOT accuracy-superiority (there is no benchmark vs a paid LLM).
13
14   ## 1. Problem & goal
15   KhelSutra's differentiator vs paid LLMs is that you can own a model trained on YOUR
footage ? your courts, your camera angles, your busy multi-court halls ? on a commercially-
clean, inspectable stack. The concrete, usable entry point already exists: rally-annotator,
a free MIT VLC Lua extension that turns "watch a match" into "produce golden labels" `[verified
README.md:22-24]`. What's missing is the education/positioning surface that tells this story
and the honest documentation of the end-to-end loop. "Good" = a viewer understands the moat,
can install the annotator and produce labels today, and is never misled into thinking one-
click training exists. Read first: `rally-annotator/README.md`, `rally-
annotator/docs/CSV_FORMAT.md`, `badminton-highlight-indexer/backend/worker/__main__.py`,
`khelsutra/docs/PER_RALLY_SELECTION.md`, `khelsutra/site/public/index.html` (the honesty voice).
16
17   ## 2. Decisions locked
18   | # | Decision | Source / date | Implication |
19   |---|-----|-----|-----|
20   | D1 | Marketing page ships NOW; SPA screen is design-tracked + owner-gated | This
synthesis 2026-06-20 | Convert surface first; operate surface deferred |
21   | D2 | ONE marketing page, three anchored sections (Annotate/Train/Setup) | Avoid thin
pages; annotator is external | Clone `capture.html` skeleton; no build step |
22   | D3 | Annotator docs live on marketing; SPA only links out | `[verified]` annotator is
a standalone VLC tool | Do not re-author VLC instructions in the SPA |
23   | D4 | NO "Train now" button/endpoint in the SPA | `[verified]` training is offline
`backend/worker train` | SPA's honest role = correction loop ? grow corpus |
24   | D5 | Frame USP as ownership/licensing/privacy/consent, NOT accuracy | `[verified]` no
user-model-vs-LLM benchmark exists | No accuracy-superiority copy anywhere |
25   | D6 | Paid LLMs never a training source; MIT/Apache/BSD only; minors excluded; opt-in |
`[verified CLAUDE.md]` + `harness.py:178,189-192` attestation | Reuse the site's existing minors
+ opt-in wording (`index.html:342-349`) |
26   | D7 | Annotator CSV needs NO transform for the engine label path | `[verified]` $4.2
schema bridge | Document the MD5+stem join as the only real prereq |
27
28   ## 3. Foundation ? research / prior art `[verified this session]`
29   The annotator (real, shipping). rally-annotator is a single-file VLC Lua
extension (`vlc/rally_annotator.lua`, VERSION 1.6.4), MIT-licensed, public at
https://github.com/Khelsutra/rally-annotator. It is button-driven (no listener capabilities;
deliberate ? VLC pause-listener is flaky on macOS / broken in 4.0), reads playback time via
`vlc.var.get(input,'time')` (micros/1e6?s, lines 201-207), and appends one row per rally to
`<video-stem>.rallies.csv` next to the video with atomic rewrite (save_all 310-335). Targets 5
net-separated racquet sports sharing one error taxonomy. CI runs a Lua 5.1 dialog suite +
layout-snapshot golden diff. VLC 3.0.x only (4.0 changed the Lua API).
30
31   The train loop (partly real). The engine's `backend/worker train` (cmd_train,
`__main__.py:231`) pulls `labels/` (+ `videos/` for the detector source), builds per-video
trajectory CSVs, assembles a manifest, shells out to `python -m training.gen0.harness` (the

```

one-way train/serve wall, `training/README.md:9-11`), and pushes a versioned bundle to `weights/<version>/`. Two trajectory sources, identical downstream: ****synth**** (DEFAULT, GPU-free, a CPU mock that ENCODES the labels ? plumbing-meaningful only, `stub\_runner.py:124-149,131-133`) and ****detector**** (real WASB, M3.5, ****GPU-gated, IN-FLIGHT**** ? the Linux-native runner does not exist yet, `02:15`, `04:98-113`). The harness fits a ~1M-param LogisticRegression on CPU and emits the artifact bundle (`weights.json` + reviewable `manifest.json`); ****serving** consumes only the bundle, never a code import** (ADR-008). The ****ship-gate** (`gate\_verdict`, `harness.py:98`) currently refuses a commercial ship** regardless of F1: WASB checkpoint provenance C3 unresolved (commercial ship refused) + F1@tIoU not yet calibrated (#172).

32
33 ****Guardrails** (verbatim, non-negotiable). ****Paid LLMs** (Gemini/OpenAI/Anthropic) = inference/serving/fallback ONLY, ****never a training data source**** (bundle attests `no\_paid\_llm\_training\_labels:True`, `harness.py:192`); label source must be human (annotator golden CSVs, `harness.py:178`). Commercial-clean deps only ? ****MIT/Apache-2.0/BSD****. Owned base = Qwen3-VL (Apache) primary; Gemma-4 AMBER/bench-only; Gemma 3 rejected. ****Minors excluded**** from the training pool; per-user training data needs ****explicit opt-in**** `[verified CLAUDE.md]`.

34
35 **## 4. Design / architecture**
36

37 **### 4.1 The annotate ? CSV ? train ? model flow** `[verified except where noted]`
38 1. ****ANNOTATE**** (rally-annotator, VLC): per rally, Mark START (serve-contact) ? Mark END (shuttle-dead) ? Ending reason ? optional shots ? Save Rally ? appends a row to `<video>.rallies.csv`. No compute, no GPU, no LLM.

39 2. ****STAGE****: `python -m src.cli.curate import-local --sport badminton --video-path match.mp4 --annotations-path match.rallies.csv`, and/or populate a corpus prefix `videos/labels/` manifest.json weights/ outputs/<video_id>/`.

40 3. ****TRAIN****: default `python -m backend.worker train --corpus-uri ? --out-uri ? --version 0.1.0` (synth, GPU-free CPU mock, ?2 labeled videos); real = add `--trajectory-source detector --segmenter wasb\_hybrid` with `indexing.detector\_impl=native` (GPU + native WASB + `wasb\_badminton\_best.pth.tar`, IN-FLIGHT).

41 4. ****HARNESS ? BUNDLE****: parse trajectory + load GT ? frame features ? honest LOVO eval ? fit final LR + tune threshold ? ship-gate ? emit `weights.json` + `manifest.json`.

42 5. ****USE**** `[researched/design, owner-gated]`: if Gen-0 clears the ship-gate A/B, the bundle drives the existing record?upload?reel loop. No per-user serving UI; no Studio Train button.

43
44 **### 4.2 The schema bridge ? annotator CSV ? engine label CSV** (MATCHES, no transform)
45 Annotator writes header

`rally\_number,start\_time,end\_time,ending\_reason,sport,shots\_count` (`rally\_annotator.lua:128`; CSV_FORMAT.md:6), `%.3f` decimal seconds, end>start. The engine reads ****by column name**** via two readers, both satisfied:

46 - ****Default synth path**** ? `synth\_trajectory\_from\_labels` uses `csv.DictReader` reading literally `row["start\_time"]`/`row["end\_time"]` (`stub\_runner.py:139`) ? ****MATCHES exactly****; extra columns ignored. (This path needs the `\*\_time` names specifically ? the short `start/end` alias is NOT accepted here.)

47 - ****Canonical GT reader**** ? `load\_gt\_intervals` accepts `start`/`end` OR `start\_time`/`end\_time`, ignores other columns, also accepts headerless positional col0,col1, tolerates decimal-seconds or MM:SS/HH:MM:SS, BOM, blank/`#` lines (`gt\_loader.py:14-90`) ? ****MATCHES**** the `\*\_time` branch.

48
49 The worker's own inference CSV uses `start,end,shot\_count,ending\_reason` (`\_\_main\_\_.py:62-74`) ? order/short-name differ, but order never matters and `gt\_loader` accepts both aliases; the annotator's `\*\_time` form is the ****safer**** of the two. ****The ONE real prerequisite is an IDENTITY/JOIN constraint, not a schema gap:**** training keys video?label by ****file MD5**** and stem-matches `labels/<name>.rallies.csv` ? `videos/<name>.<ext>` (`\_\_main\_\_.py:271-280`; `02:48-60`). Stems must agree and the staged video bytes must be ****byte-identical**** to what was annotated ? any re-encode changes the MD5 and breaks the join (collector manifest currently omits `md5`, M2 prereq, `02:62-68`).

50
51 **### 4.3 Reuse map**

52 - ****Marketing**** `[verified]`: clone `site/public/capture.html` skeleton + brand tokens (`index.html:25-32`); reuse `.honest`/`.notes` components (`:152-158`, used `:351-354`, `:400-415`); add a `own-model` alias in `site/server.js:49-51` (Worker `env.ASSETS.fetch` needs no change, `site/src/index.js:84`); cross-link nav `:218-223`, the "gets sharper" section

```

`327-356`, footer `528`; screenshots under `site/public/img/` (precedent `site/og.png` +
`make_og.py`).
53 - SPA `[design, gated]`: today `web/src/App.tsx` is one static screen, no router
(`web/package.json`). A correction screen needs react-router-dom (or a hash switch) + the typed
client `web/src/api/client.ts`/`types.ts` + `web/src/lib/log.ts`; it realizes P10 in
`PER_RALLY_SELECTION.md:122` and depends on a new engine endpoint `PATCH /api/segments/{id}`
(dead columns `candidate_segments.human_label/notes/confirmed_segment_id`).
54 - Annotator content `[verified]`: embed CSV example + install commands + ending-
reason table verbatim from the repo; link, don't re-author.
55
56 ## 5. Threat model / privacy
57 - Footage stays the user's: the annotator runs locally in the user's own VLC and
writes a CSV next to the video ? labeling happens with footage never leaving the machine
`[verified]`.
58 - Training use = explicit opt-in, never automatic `[verified CLAUDE.md]`; reuse the
site's existing opt-in wording.
59 - Minors excluded from any training pool `[verified CLAUDE.md + live site "Minors
are protected"]` ? keep prominent (audience = academics with children).
60 - No paid-LLM laundering: the model never trains on a paid LLM's output
(terms/copyright), attested in the bundle (`harness.py:192`).
61 - Residual: base-model (Qwen3-VL) pretraining provenance is unauditale
`[researched]` ? owner risk-acceptance required before VLM "train your own model" copy goes
beyond roadmap.
62
63 ## 6. Honest limits ? what this does NOT do
64 A shipped marketing page does NOT mean training is one-click (it is an offline CLI),
does NOT mean a real model exists (synth = mock; real = GPU + in-flight native WASB + owner-
gated ship-gate), and does NOT imply any accuracy claim vs a paid LLM (no benchmark exists). The
annotator is a separate VLC 3.0.x desktop plugin, not in-browser; it is click-driven (no
marking hotkeys). The SPA correction screen is design-tracked + owner-gated and depends on
an unbuild `PATCH /api/segments` endpoint. Two CSV schemas exist (annotator lighter vs Roora
vendor) ? do not conflate.
65
66 ## 7. Deliverables & progress tracker ? source of truth
67 Legend: ? Todo · ? In progress · ? Done · ? Gated. One small PR per row, branched from
`origin/master`, no stacking.
68
69 | ID | Deliverable | Depends on | Gated? | Status | PR |
70 |----|-----|-----|-----|-----|----|
71 | P0 | This tracked doc in `kheutra/docs/`, indexed in `docs/README.md` | ? | No | ? |
[15](https://github.com/avidullu/kheutra/pull/15) |
72 | P1 | `site/public/own-model.html` (clone capture.html; 3 sections; honesty fences;
placeholder shots) | P0 | No | ? | (this PR) |
73 | P2 | `own-model` alias in `site/server.js` (Worker needs no change) | P1 | No | ? |
(this PR) |
74 | P3 | Cross-links: nav + footer (the "gets sharper" section link is a nice-to-have,
deferred) | P1 | No | ? | (this PR) |
75 | P4 | Embed verbatim repo text (CSV/install/ending-reasons) + GitHub link | P1 | No | ?
| (this PR) |
76 | P5 | Capture + commit real screenshots (consented, no minors, faces redacted); replace
placeholders | P1 | ? Owner (footage) | ? | ? |
77 | P6 | Owner: slug + 1-vs-3-page + train-positioning honesty | ? | ? Owner | ? | ? |
78 | P7 | [design] SPA correction/contribute route (RallyReview + link-out) | P8 | ? Owner
| ? | ? |
79 | P8 | [design] Engine `PATCH /api/segments/{id}` (M4 corpus extension) | ? | ? Owner
(engine) | ? | ? |
80 | P9 | [researched] Serve a shipped Gen-0 bundle into record?reel | native WASB + ship-
gate | ? Owner+blocked | ? | ? |
81
82 ## 8. Open questions ? owner / external
83 Blocking gates: (1) Greenlight how prominently to position "train your own model"
(CLI + roadmap). (2) Approve the SPA correction screen + the new `PATCH /api/segments/{id}`
endpoint. (3) Owner risk-acceptance on Qwen3-VL provenance before VLM copy. (4) Provide
consented no-minors footage for the 5 screenshots.
84 Nice-to-knows: slug (`own-model` vs `byo-model` vs `annotate`); one page vs

```

```

three; react-router-dom vs hash switch; cross-link `docs/LOCAL_APPLIANCE_ARCHITECTURE.md` once
PR #13 merges.
85
86     ## 9. Definition of done
87     - [ ] `own-model.html` live with Annotate/Train/Setup sections, honesty fences, verbatim
repo text, GitHub link, real (non-fabricated) screenshots.
88     - [ ] `/own-model` alias works on both the Node host and the Cloudflare Worker; cross-
linked from nav/section/footer.
89     - [ ] Every claim tagged `[verified]`/[design]`/[researched]`; no accuracy-superiority
claim; no "Train now" button; VLC-3.0.x-only + GPU/native-WASB-gated + owner-gated stated
plainly.
90     - [ ] Schema-bridge + MD5/stem-join prerequisite documented; the two CSV schemas kept
distinct.
91     - [ ] $7 tracker reflects real shipped state; doc indexed in `docs/README.md`.
92
93     ## 10. References
94     **Annotator `[verified]`**: **`rally-annotator/README.md`, `vlc/rally_annotator.lua`
(HEADER :128; now_seconds :201-207; marks :608-628; save_all :310-335; resolve_out_path
:253-267), `docs/CSV_FORMAT.md`, `docs/ENDING_REASONS.md`, `LICENSE` (MIT),
`.github/workflows/ci.yml`.
95     **Engine `[verified]`**: **`backend/worker/__main__.py` (cmd_train :231;
_write_intervals_csv :62-74; join :271-280; harness shell-out :321-328),
`backend/pipeline/detectors/stub_runner.py` (synth :124-149, :131-133, :139),
`training/gen0/harness.py` (gate_verdict :98; label_source :178; attestation :189-192),
`backend/eval/gt_loader.py` (:14-90), `docs/SETUP_NEW_MACHINE.md`,
`docs/DECOUPLED_COMPUTE/02-COMPUTE-MAP-AND-CONTRACT.md`, `docs/DECOUPLED_COMPUTE/04-BUILD-AND-
TEST-PLAN.md`, `docs/OWNED_MODEL_IMPLEMENTATION_PLAN.md`, `CLAUDE.md`.
96     **Khe1Sutra `[verified]`**: **`site/public/index.html`, `site/public/capture.html`,
`site/server.js`, `site/src/index.js`, `web/src/App.tsx`, `web/package.json`, `docs/README.md`,
`docs/PER_RALLY_SELECTION.md`.
97
98     ### Changelog
99     - 2026-06-20 ? Initial DRAFT consolidating annotator + train-loop + UI-surface research
into one tracked doc (multi-agent design pass; schema-bridge + no-images + train-guardrails re-
verified against source before writing).
100

```

23. assistant (2026-06-21T13:35:31.220Z)

This doc is extremely thorough and corroborates the engine facts. Let me verify a few load-bearing claims directly and check the regression gate (the product-side admission gate), the per_user gates, and how a trained bundle is actually served. Let me also verify there's no in-browser annotation in the SPA and check the API surface.

24. user (2026-06-21T13:35:32.655Z)

```

1     """Per-user promotion gate ? the safety rail for flag-gated personalization.
2
3     The owner adopted personalization as a **flag-gated** feature on a **generalization-
first**
4     baseline: a cue/variant B may be flipped ON *for one user* only when that user's own
5     held-out evidence honestly clears the same small-N bar the global eval uses ? otherwise
we
6     fall back to the shipped baseline (variant A). This module is the decision logic for
that
7     flip; it is pure (no I/O) and **reuses** ``backend.eval.eval_stats`` for every statistic
8     (``paired_delta_ci`` + ``paired_sign_test``) so the math is never reinvented.
9
10    The counter-case this guards against (and its mitigations, implemented here):
11
12    1. **Minimum-N floor.** Personalizing off 1-2 of a user's videos is the small-N-lies
regime
13    (see ``eval_stats`` C1). Refuse to promote below ``min_n`` videos ? point estimates
from
14    too few held-out videos are noise, not signal.

```

```

15     2. **Honest lift, not a single mean.** Promote only when the paired ?F1 95% bootstrap CI
16     **excludes 0** *and* the paired sign test agrees (majority of videos win, p <=
threshold).
17     This is the global promotion bar applied per user.
18     3. **Structural guards are never disabled on "no lift".** A *structural* guard (e.g.
Gate-A
19     net-crossings ? the held-shuttle false-positive killer) earns its keep through
failure
20     modes a given user's corpus may not contain yet. Measuring "no per-user lift" on a
corpus
21     that simply lacks those failure cases must NEVER turn the guard OFF. For
``structural=True``
22     the decision is therefore **always "keep ON"**, independent of the measured delta.
23
24     Below the floor / insufficient evidence ? fall back to the shipped baseline (the
conservative
25     default), never an unsafe promotion. All additive, pure, deterministic (the bootstrap
seed is
26     threaded through). Nothing here changes existing eval behaviour; callers opt in.
27     ""
28     from __future__ import annotations
29
30     from dataclasses import dataclass, field
31     from typing import Any, Dict, Sequence
32
33     from backend.eval.eval_stats import paired_delta_ci, paired_sign_test
34
35     __all__ = ["PromotionDecision", "should_promote_for_user"]
36
37
38     @dataclass
39     class PromotionDecision:
40         """Structured result of a per-user promotion decision.
41
42         - ``promote`` ? flip variant B ON for this user (only ever ``True`` for a non-
structural
43         cue that cleared the full bar).
44         - ``keep_on`` ? the cue/guard should be ON for this user. For a structural guard
this is
45         always ``True`` (it is never disabled on "no lift"); for a non-structural cue it
equals
46         ``promote``.
47         - ``reason`` ? short human-readable explanation of the decision.
48         - the remaining fields surface the evidence (n, mean delta, CI bounds and the two
tests)
49         so a caller can log/audit exactly why a user was or wasn't personalized.
50         ""
51
52         promote: bool
53         keep_on: bool
54         reason: str
55         n: int
56         mean_delta: float
57         ci_low: float
58         ci_high: float
59         ci_excludes_zero: bool
60         sign_test: Dict[str, float] = field(default_factory=dict)
61         floor_met: bool = False
62         structural_protected: bool = False
63
64         def as_dict(self) -> Dict[str, Any]:
65             """Plain-dict view (for JSON logging / telemetry)."""
66             return {
67                 "promote": self.promote,
68                 "keep_on": self.keep_on,

```

```

69         "reason": self.reason,
70         "n": self.n,
71         "mean_delta": self.mean_delta,
72         "ci_low": self.ci_low,
73         "ci_high": self.ci_high,
74         "ci_excludes_zero": self.ci_excludes_zero,
75         "sign_test": self.sign_test,
76         "floor_met": self.floor_met,
77         "structural_protected": self.structural_protected,
78     }
79
80
81     def should_promote_for_user(
82         a_scores: Sequence[float],
83         b_scores: Sequence[float],
84         *,
85         structural: bool = False,
86         min_n: int = 5,
87         p_threshold: float = 0.05,
88         confidence: float = 0.95,
89         n_boot: int = 2000,
90         seed: int = 0,
91         eps: float = 0.0,
92     ) -> PromotionDecision:
93         """Decide whether to flip variant B ON for ONE user from that user's held-out
94         scores.
95         ``a_scores`` / ``b_scores`` are the user's **per-video** held-out F1 (or any score)
96         for
97         variant A (control / shipped baseline) and variant B (with the cue), aligned by
98         video
99         (same order). The decision rules (faithful to the module docstring):
100
101         1. ``structural=True`` ? the guard is **never disabled on "no lift"**: return
102           ``keep_on=True, promote=False`` regardless of the measured per-user delta. Its
103           value is
104           in failure modes the user's corpus may not yet contain.
105
106         2. Otherwise promote **iff ALL** hold ? the minimum-N floor (``n >= min_n``), the
107           paired
108           ?F1 bootstrap CI **excludes 0**, and the paired sign test agrees (majority of
109           videos
110           win, ``p_value <= p_threshold``). Any miss ? fall back to the shipped baseline.
111
112         Statistics come straight from ``eval_stats`` (``paired_delta_ci`` /
113         ``paired_sign_test``);
114         deterministic given ``seed``. Returns a :class:`PromotionDecision`.
115         """
116         n = min(len(a_scores), len(b_scores))
117         floor_met = n >= min_n
118
119         # --- Evidence (reuse eval_stats; never reinvent the statistics)
120         -----
121         if n == 0:
122             # No paired videos at all ? nothing to test. Surface an empty-but-honest result.
123             sign_test: Dict[str, float] = paired_sign_test([], eps)
124             mean_delta, ci_low, ci_high, ci_excludes_zero = 0.0, 0.0, 0.0, False
125         else:
126             delta = paired_delta_ci(
127                 a_scores, b_scores,
128                 confidence=confidence, n_boot=n_boot, seed=seed, eps=eps,
129             )
130             mean_delta = float(delta["mean_delta"])
131             ci_low = float(delta["ci_low"])
132             ci_high = float(delta["ci_high"])
133             ci_excludes_zero = bool(delta["ci_excludes_zero"])

```



```

126         sign_test = delta["sign_test"]
127
128     sign_p = float(sign_test.get("p_value", 1.0))
129     sign_wins = float(sign_test.get("wins", 0.0))
130     sign_losses = float(sign_test.get("losses", 0.0))
131     # The sign test "agrees" with a promotion only if it is significant AND the majority
132     # leans the *winning* way (more B-wins than B-losses) ? a significant net *loss*
must
133     # never satisfy the promotion bar.
134     sign_agrees = (sign_p <= p_threshold) and (sign_wins > sign_losses)
135
136     # --- Rule 1: structural guards are never disabled on "no lift"
-----
137     if structural:
138         return PromotionDecision(
139             promote=False,          # nothing to "promote" ? it ships ON by design
140             keep_on=True,           # ALWAYS kept ON, regardless of measured per-user
lift
141             reason=(
142                 "structural guard kept ON regardless of measured per-user lift "
143                 "(value is in failure modes this user's corpus may not yet contain)"
144             ),
145             n=n,
146             mean_delta=mean_delta,
147             ci_low=ci_low,
148             ci_high=ci_high,
149             ci_excludes_zero=ci_excludes_zero,
150             sign_test=sign_test,
151             floor_met=floor_met,
152             structural_protected=True,
153         )
154
155     # --- Rule 2: minimum-N floor (refuse before even looking at lift)
-----
156     if not floor_met:
157         return PromotionDecision(
158             promote=False,
159             keep_on=False,          # fall back to the shipped baseline (variant A)
160             reason=(
161                 f"below minimum-N floor (n={n} < min_n={min_n}); "
162                 "falling back to shipped baseline"
163             ),
164             n=n,
165             mean_delta=mean_delta,
166             ci_low=ci_low,
167             ci_high=ci_high,
168             ci_excludes_zero=ci_excludes_zero,
169             sign_test=sign_test,
170             floor_met=False,
171             structural_protected=False,
172         )
173
174     # --- Rule 2 (cont.): honest lift ? CI excludes 0 AND sign test agrees
-----
175     if ci_excludes_zero and sign_agrees:
176         promote = True
177         reason = (
178             f"promoted: n={n} >= {min_n}, ?F1 95% CI "
179             f"[{ci_low:.4f}, {ci_high:.4f}] excludes 0, "
180             f"sign test {int(sign_wins)}-{int(sign_losses)} (p={sign_p:.4f} <=
{p_threshold})"
181         )
182     else:
183         promote = False
184         misses = []

```

```

185         if not ci_excludes_zero:
186             misses.append(f"?F1 95% CI [{ci_low:.4f}, {ci_high:.4f}] spans 0")
187         if not sign_agrees:
188             misses.append(
189                 f"sign test does not agree ({int(sign_wins)}-{int(sign_losses)}), "
190                 f"p={sign_p:.4f}")
191             )
192         reason = (
193             "insufficient evidence (" + "; ".join(misses) + "); "
194             "falling back to shipped baseline"
195         )
196
197     return PromotionDecision(
198         promote=promote,
199         keep_on=promote,          # non-structural: keep_on tracks promote exactly
200         reason=reason,
201         n=n,
202         mean_delta=mean_delta,
203         ci_low=ci_low,
204         ci_high=ci_high,
205         ci_excludes_zero=ci_excludes_zero,
206         sign_test=sign_test,
207         floor_met=True,
208         structural_protected=False,
209     )
210

```

25. user (2026-06-21T13:35:32.899Z)

```

1  """SERVED Gate-A verification: run the *production* ``FusionSegmenter`` decision seems
2  on a cached WASB trajectory and score the resulting AI-handoff windows vs golden labels.
3
4  The offline litmus (``backend.eval.ablation`` / ``fusion_compare``) measures Gate A on
the
5  **permissive proposal windowing** (``distill_local.PROPOSAL`` = recall-oriented) re-
scored
6  through the experiment harness's ``_gate_mask``. That validates the *signal*. It does
NOT
7  exercise the **served path**: in production the windows come from
8  ``trajectory_to_action_windows`` at the production operating point
9  (``velocity_threshold=0.02, dead_time_merge_gap=2.0, min_action_window_duration=2.0``),
the
10 net-crossing count is computed by ``FusionSegmenter._enrich_candidate_stats``, and the
gate is
11 ``FusionSegmenter._select_for_handoff``. THIS module drives exactly those production
methods ?
12 no reimplementaion of the windowing or the gate ? on a cached trajectory CSV, so the
13 verification reflects what ``process_video`` actually hands to Gemini.
14
15 GPU-free and Gemini-free: it stops at the handoff-selection boundary (the windows that
WOULD
16 be sent to the AI). With the person cue OFF (the default) nothing here loads torch or
touches
17 the GPU ? it just re-scores cached WASB trajectories.
18
19 Run (owner box, golden trajectories present):
20     python -m backend.eval.served_gate_a --manifest output/human_lovo_manifest.json
21
22 from __future__ import annotations
23
24 import argparse
25 import json
26 from dataclasses import dataclass
27 from typing import Any, Dict, List, Optional, Tuple
28

```

```

29 from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
30 from backend.pipeline.segmenters.fusion_hybrid import FusionSegmenter
31 from backend.eval.calibrate_wasb import load_golden_gts
32 from backend.eval.metrics import Interval, score, Score
33 from backend.eval import segmentation_metrics as _segm
34
35 # Production windowing operating point (config.json indexing defaults ? the values
36 # TrajectoryHybridSegmenter.process_video reads for the served fusion run).
37 PROD_VELOCITY_THRESH = 0.02
38 PROD_MERGE_GAP = 2.0
39 PROD_MIN_WINDOW = 2.0
40
41
42 def _served_idx_cfg(min_crossings: int) -> Dict[str, Any]:
43     """The ``indexing`` config dict the served FusionSegmenter seams read ? mirrors
44     config.json's ``indexing.fusion`` block, with Gate A at ``min_crossings`` and the
45     person cue OFF (so no GPU / torch is touched)."""
46     return {
47         "fusion": {
48             "substrate": "wasb",
49             "min_crossings": min_crossings,
50             "min_players": 0,
51             "cue_flags": {},
52         },
53         "wasb": {"velocity_threshold": PROD_VELOCITY_THRESH},
54         "dead_time_merge_gap": PROD_MERGE_GAP,
55         "min_action_window_duration": PROD_MIN_WINDOW,
56     }
57
58
59 def served_handoff_windows(trajecory_csv: str, fps: float, frame_width: float,
60                           min_crossings: int) -> List[Interval]:
61     """Run the PRODUCTION FusionSegmenter decision path on a cached trajectory and
62     return the
63     windows that would reach the AI handoff (i.e. the served rally candidates).
64     Uses the real ``FusionSegmenter`` seam methods ? ``_enrich_candidate_stats``
65     (computes
66     ``net_crossings``) and ``_select_for_handoff`` (Gate A) ? over windows from the
67     production
68     ``trajectory_to_action_windows`` operating point. No GPU, no Gemini: the person cue
69     is OFF.
70     """
71     points = TrackNetRunner.parse_trajectory_csv(trajecory_csv)
72     windows = TrackNetRunner.trajectory_to_action_windows(
73         points, fps=fps, frame_width=frame_width,
74         velocity_thresh=PROD_VELOCITY_THRESH, merge_gap=PROD_MERGE_GAP,
75         min_window_duration=PROD_MIN_WINDOW,
76     )
77     idx_cfg = _served_idx_cfg(min_crossings)
78     # Construct the real served segmenter (db=None, config carries the indexing block).
79     We only
80     # call the pure decision seams ? no process_video, no DB, no provider ? exactly as
81     the
82     # registry constructs it for `name`/`description`.
83     seg = FusionSegmenter(None, {"indexing": idx_cfg, "sport": "badminton"})
84     # The served per-window stats (incl. net_crossings) via the production enrichment
85     hook.
86     window_stats = [
87         seg._enrich_candidate_stats(cw, points, fps, frame_width, video_path="",
88                                   idx_cfg=idx_cfg)
89         for cw in windows
90     ]
91     keep = seg._select_for_handoff(windows, window_stats, idx_cfg)
92     return [(windows[i]["start"], windows[i]["end"]) for i in keep]

```

```

87
88
89 @dataclass
90 class ServedVideoResult:
91     name: str
92     num_gt: int
93     on: Score          # Gate-A ON (min_crossings=2)
94     off: Score         # Gate-A OFF (min_crossings=0, control)
95     seg_ratio_on: float # |preds| / |gts| with Gate-A ON (over-segmentation)
96     seg_ratio_off: float
97     n_handoff_on: int
98     n_handoff_off: int
99
100 @property
101 def delta_f1(self) -> float:
102     return self.on.f1 - self.off.f1
103
104
105 def evaluate_video(spec: Dict[str, Any], iou: float = 0.5,
106                  on_crossings: int = 2) -> ServedVideoResult:
107     """Score the served Gate-A ON (min_crossings=on_crossings) vs OFF (0) handoff
108     windows for
109     one golden video against its golden labels."""
110     fps, fw = float(spec["fps"]), float(spec["frame_width"])
111     traj, labels = str(spec["trajectory"]), str(spec["labels"])
112     gts = load_golden_gts(labels)
113     preds_on = served_handoff_windows(traj, fps, fw, min_crossings=on_crossings)
114     preds_off = served_handoff_windows(traj, fps, fw, min_crossings=0)
115     return ServedVideoResult(
116         name=str(spec["name"]),
117         num_gt=len(gts),
118         on=score(preds_on, gts, iou),
119         off=score(preds_off, gts, iou),
120         seg_ratio_on=_segm.segment_count_ratio(preds_on, gts),
121         seg_ratio_off=_segm.segment_count_ratio(preds_off, gts),
122         n_handoff_on=len(preds_on),
123         n_handoff_off=len(preds_off),
124     )
125
126 def evaluate_manifest(specs: List[Dict[str, Any]], iou: float = 0.5,
127                    on_crossings: int = 2) -> List[ServedVideoResult]:
128     return [evaluate_video(s, iou, on_crossings) for s in specs]
129
130
131 def aggregate(results: List[ServedVideoResult]) -> Dict[str, Any]:
132     """Mean ON/OFF F1 + over-seg ratio, the paired ?F1, and the per-video win count."""
133     n = len(results)
134
135     def _mean(getter) -> float:
136         return sum(getter(r) for r in results) / n if n else 0.0
137
138     deltas = [r.delta_f1 for r in results]
139     wins = sum(1 for d in deltas if d > 1e-9)
140     losses = sum(1 for d in deltas if d < -1e-9)
141     return {
142         "num_videos": n,
143         "mean_f1_on": _mean(lambda r: r.on.f1),
144         "mean_f1_off": _mean(lambda r: r.off.f1),
145         "mean_delta_f1": _mean(lambda r: r.delta_f1),
146         "mean_precision_on": _mean(lambda r: r.on.precision),
147         "mean_precision_off": _mean(lambda r: r.off.precision),
148         "mean_recall_on": _mean(lambda r: r.on.recall),
149         "mean_recall_off": _mean(lambda r: r.off.recall),
150         "mean_seg_ratio_on": _mean(lambda r: r.seg_ratio_on),

```

```

151         "mean_seg_ratio_off": _mean(lambda r: r.seg_ratio_off),
152         "wins": wins,
153         "losses": losses,
154     }
155
156
157 def format_table(results: List[ServedVideoResult], agg: Dict[str, Any], iou: float) ->
str:
158     lines = [
159         f"=== SERVED Gate-A verification (production FusionSegmenter path, IoU>={iou})
===",
160         f"  {'video':<24s} {'gt':>3s} {'F1_off':>7s} {'F1_on':>7s} {'?F1':>7s} "
161         f"{'segR_off':>9s} {'segR_on':>8s} {'hand_off':>9s} {'hand_on':>8s}",
162     ]
163     for r in results:
164         lines.append(
165             f"  {r.name:<24s} {r.num_gt:>3d} {r.off.f1:>7.3f} {r.on.f1:>7.3f} "
166             f"{r.delta_f1:>+7.3f} {r.seg_ratio_off:>9.2f} {r.seg_ratio_on:>8.2f} "
167             f"{r.n_handoff_off:>9d} {r.n_handoff_on:>8d}")
168     lines += [
169         f"  {'-' * 92}",
170         f"  MEAN  F1 {agg['mean_f1_off']:.3f} (off) -> {agg['mean_f1_on']:.3f} (on)  "
171         f"?F1={agg['mean_delta_f1']:+.3f}  "
172         f"P {agg['mean_precision_off']:.3f}->{agg['mean_precision_on']:.3f}  "
173         f"R {agg['mean_recall_off']:.3f}->{agg['mean_recall_on']:.3f}",
174         f"  over-seg ratio {agg['mean_seg_ratio_off']:.2f} (off) -> "
175         f"{agg['mean_seg_ratio_on']:.2f} (on)  | wins {agg['wins']}/{agg['num_videos']}
"
176         f"(losses {agg['losses']})",
177     ]
178     return "\n".join(lines)
179
180
181 def run(manifest_path: str, iou: float = 0.5) -> Tuple[List[ServedVideoResult],
Dict[str, Any]]:
182     with open(manifest_path, encoding="utf-8") as f:
183         specs = json.load(f)
184         results = evaluate_manifest(specs, iou=iou)
185         return results, aggregate(results)
186
187
188 def main(argv: Optional[List[str]] = None) -> None:
189     ap = argparse.ArgumentParser(
190         description="Verify served Gate-A on real footage via the production
FusionSegmenter path.")
191     ap.add_argument("--manifest", default="output/human_lovo_manifest.json")
192     ap.add_argument("--iou", type=float, default=0.5)
193     args = ap.parse_args(argv)
194     results, agg = run(args.manifest, iou=args.iou)
195     print(format_table(results, agg, args.iou))
196
197
198 if __name__ == "__main__":
199     main()
200

```

26. assistant (2026-06-21T13:35:36.637Z)

Now let me verify the API surface (is there any annotate/train endpoint?), check how a trained bundle gets served (model loading), and confirm the SPA has no in-browser annotation. Let me also check the curate import-local CLI referenced in the train flow.

27. user (2026-06-21T13:35:38.731Z)

```
backend\api\uploads.py:20:from fastapi import APIRouter, HTTPException, Request
```

```
backend\api\uploads.py:26:router = APIRouter()
backend\api\uploads.py:64:@router.post("/api/upload/init")
backend\api\uploads.py:89:@router.put("/api/upload/{upload_id}/part/{index}")
backend\api\uploads.py:123:@router.post("/api/upload/{upload_id}/complete")
backend\api\uploads.py:147:@router.delete("/api/upload/{upload_id}")
```

28. user (2026-06-21T13:35:38.799Z)

```
backend\api\__init__.py
backend\api\uploads.py
backend\api\job_worker.py
backend\api\auth.py
```

29. user (2026-06-21T13:35:40.136Z)

```
86:     def to_dict(self) -> dict:
98:     def from_dict(cls, d: dict) -> "LogisticRegression":
109:         json.dump(self.to_dict(), f, indent=2)
114:         return cls.from_dict(json.load(f))
```

30. assistant (2026-06-21T13:35:44.006Z)

The API routers are split. Let me find the main server route registration to see the full endpoint list, and check whether any trained bundle is actually consumed at serving time (is there a code path that loads weights.json into a segmenter?).

31. user (2026-06-21T13:35:45.532Z)

```
65:os.makedirs("backend/api/static", exist_ok=True)
66:app.mount("/static", StaticFiles(directory="backend/api/static"), name="static")
68:@app.get("/", response_class=HTMLResponse)
70:    index_path = "backend/api/static/index.html"
74:    return "<h3>Sports Highlight Indexer Server is Running. Please create index.html in
backend/api/static/</h3>"
138:@app.get("/api/config")
142:@app.get("/api/health")
158:    This is the former synchronous /api/process body, unchanged in behavior, now run on
275:@app.post("/api/process", status_code=202)
277:    """Submit a processing job (202 + job_id). Poll GET /api/jobs/{job_id} for state.
322:        content={"job_id": job_id, "status": "queued", "poll": f"/api/jobs/{job_id}"},
326:@app.get("/api/jobs/{job_id}/cost")
341:@app.get("/api/videos/{video_id}/cost")
352:@app.get("/api/jobs/{job_id}")
380:app.include_router(uploads_api.router)
385:@app.get("/api/highlights/{video_id}/{filename}")
387:    """Stream a compiled highlight reel ? `filename` is the bare name given to /api/compile
404:        detail=f"No compiled file named '{name}'. Compile first via
/api/compile.")
409:@app.post("/api/query")
420:@app.post("/api/compile")
474:    download_url = f"/api/highlights/{quote(req.video_id)}/{quote(out_name)}"
643:    # /api/compile ? reads the same value instead of a write-only runtime hack.
```

32. user (2026-06-21T13:35:47.454Z)

```
Found 9 files
backend\pipeline\detectors\native_wasb_runner.py
backend\pipeline\segmenters\fusion_hybrid.py
backend\pipeline\detectors\stub_runner.py
backend\pipeline\detectors\wasb_runner.py
backend\pipeline\detectors\base.py
backend\pipeline\personalization\serving.py
backend\pipeline\detectors\person_detector.py
backend\pipeline\detectors\audio_features.py
```

backend\pipeline\detectors\fusion_features.py

33. assistant (2026-06-21T13:35:50.792Z)

Let me check the personalization/serving.py to see if a trained bundle is actually consumed at serving time, and confirm there's no annotate/train endpoint in the API (the endpoints are upload/process/query/compile/jobs only). Let me also confirm the SPA has no annotation feature.

34. user (2026-06-21T13:35:51.277Z)

```
1      """Per-user serving bridge ? resolve a user's fusion config + classifier from the v5
store.
2
3      PERSONALIZATION_PLAN.md $2 (Phase 1, the "#1 wall"): a personalized model is a per-user
4      ``Variant`` (cue_flags + ``min_crossings`` / ``min_players`` + an optional
``LogisticRegression``
5      + threshold). It does NOT require a segmenter rewrite ? it is a per-user
``FusionConfig`` overlay
6      resolved UPSTREAM of detection, plus a classifier the decision seam re-scores persisted
features
7      with. This module is that resolution step, and only that step.
8
9      What this module does (additive, pure-ish ? one DB read, no writes):
10     - ``resolve_user_fusion_config`` ? overlay the active ``user_models`` row's
``fusion_config``
11     onto the served fusion config dict, returning the config the served path WOULD use.
Falls
12     back to the served baseline (a copy, unchanged) when there is no active model, or
the model
13     is below the promotion-confidence floor (``n_videos_at_fit < min_n``), or it carries
no
14     fusion overlay. The min-N floor here is the SAME ``per_user_gate`` promotion floor:
below it
15     a per-user A/B is not trusted enough to override a baseline default (PLAN $1.3).
16     - ``load_user_classifier`` ? rehydrate the stored ``classifier_json`` into a
17     ``LogisticRegression`` via ``from_dict`` (None when the user has no active model or
it is a
18     config-only model). ``LogisticRegression.load`` is never called in
``backend/pipeline/``
19     today ? this is the load half of the serving bridge.
20
21     ? NOT WIRED INTO THE SERVED PATH. ``fusion_hybrid._select_for_handoff`` still reads only
the
22     static ``idx_cfg["fusion"]`` ? it does NOT call anything here. Wiring this resolver into
that
23     seam (so a pinned, promoted per-user model actually changes the served handoff) is the
NEXT,
24     owner-gated PR, because it is the first change to served behavior. Until then this
is pure
25     groundwork: import it, unit-test it, but nothing in production reads it. With NULL
``user_id``
26     (the local install) and no active model, ``resolve_user_fusion_config`` returns the
served
27     config unchanged ? byte-identical to today.
28     """
29     from __future__ import annotations
30
31     import copy
32     from typing import Any, Dict, Optional
33
34     from backend.eval.classifier import LogisticRegression
35
36     __all__ = [
37         "DEFAULT_MIN_N",
38         "resolve_user_fusion_config",
```

```

39         "load_user_classifier",
40     ]
41
42     #: Promotion-confidence floor (PERSONALIZATION_PLAN.md §1.3, owner-confirmed ``min_n =
43     5``).
44     #: A per-user model fit on fewer than this many of the user's videos is NOT trusted
45     enough to
46     #: override a baseline fusion default ? we fall back to the served baseline. This
47     mirrors
48     #: ``backend.eval.per_user_gate.should_promote_for_user``'s ``min_n`` default.
49     DEFAULT_MIN_N = 5
50
51     def resolve_user_fusion_config(
52         db: Any,
53         user_id: Optional[str],
54         served_cfg: Dict[str, Any],
55         *,
56         min_n: int = DEFAULT_MIN_N,
57     ) -> Dict[str, Any]:
58         """Resolve the fusion config the served path should use for ``user_id``.
59
60         Overlays the active ``user_models`` row's ``fusion_config`` (a partial per-user
61         ``FusionConfig`` dict ? e.g. raised ``min_crossings`` / ``min_players`` / flipped
62         ``cue_flags``) onto a COPY of ``served_cfg`` (the baseline served fusion config
63         dict, i.e.
64         ``idx_cfg["fusion"]``). The original ``served_cfg`` is never mutated.
65
66         Falls back to the served baseline (an unmodified deep copy) when ANY of:
67         - the user has no active model;
68         - the active model was fit below the promotion floor (``n_videos_at_fit < min_n``)
69         ? too
70         little of the user's own footage to trust a per-user override (PLAN §1.3);
71         - the active model carries no ``fusion_config`` overlay (a classifier-only model).
72
73         So a brand-new / NULL (local) user with no active model gets the served baseline
74         unchanged
75         ? never denied service, never altered. The overlay is a shallow key-level merge:
76         per-user
77         keys win, every key the user did not set keeps the baseline value.
78         """
79         base = copy.deepcopy(served_cfg)
80         model = db.get_active_user_model(user_id)
81         if not model:
82             return base
83         if int(model.get("n_videos_at_fit") or 0) < min_n:
84             # Below the trust floor ? keep the baseline (the conservative default).
85             return base
86         overlay = model.get("fusion_config")
87         if not overlay:
88             return base # config-only fallback: a classifier-only model has nothing to
89         overlay
90         base.update(overlay)
91         return base
92
93     def load_user_classifier(db: Any, user_id: Optional[str]) ->
94     Optional[LogisticRegression]:
95         """Rehydrate the active per-user classifier for ``user_id``, or None.
96
97         Returns a ``LogisticRegression`` rebuilt from the stored ``classifier_json`` (via
98         ``from_dict``) when the user has an active model that carries one; None when there
99         is no
100         active model or it is a config-only (no-classifier) model. The caller (the FUTURE
101         served

```



```

93         wiring) would feed this model's ``predict_proba`` over the persisted
``window_stats`` ?
94         the same math ``experiment.predict`` runs offline. Nothing calls this on the served
path
95         yet; it is the load half of the serving bridge.
96         """
97         model = db.get_active_user_model(user_id)
98         if not model:
99             return None
100         blob = model.get("classifier_json")
101         if not blob:
102             return None
103         return LogisticRegression.from_dict(blob)
104

```

35. user (2026-06-21T13:35:53.064Z)

Found 14 files

```

C:\Users\avidu\Projects\khehsutra\web\src\i18n\hygiene.test.ts
C:\Users\avidu\Projects\khehsutra\web\src\i18n\locales\en\common.json
C:\Users\avidu\Projects\khehsutra\web\src\i18n\locales\hi\common.json
C:\Users\avidu\Projects\khehsutra\web\src\i18n\locales\kn\common.json
C:\Users\avidu\Projects\khehsutra\web\src\components\IngestPanel.tsx
C:\Users\avidu\Projects\khehsutra\web\src\components\IngestPanel.test.tsx
C:\Users\avidu\Projects\khehsutra\web\src\components\LanguageSwitcher.tsx
C:\Users\avidu\Projects\khehsutra\web\src\components\CompileResult.tsx
C:\Users\avidu\Projects\khehsutra\web\src\components\QueryBar.tsx
C:\Users\avidu\Projects\khehsutra\web\src\components\RallyGrid.tsx
C:\Users\avidu\Projects\khehsutra\web\src\components\SelectionFooter.tsx
C:\Users\avidu\Projects\khehsutra\web\src\components\QueryBar.examples.test.ts
C:\Users\avidu\Projects\khehsutra\web\src\lib\query.ts
C:\Users\avidu\Projects\khehsutra\web\src\lib\query.test.ts

```

36. assistant (2026-06-21T13:35:59.164Z)

The grep hits in the SPA are mostly from i18n/query matches on words like "label"/"train" (e.g. "training"/"label" in copy). Let me confirm there's no actual annotation UI by checking what the SPA components do, and verify the train CLI prereqs (the dual-env / WASB native runner status) by reading the native runner healthcheck and the DECOUPLED compute doc references. Let me also check the curate import-local CLI in sports-data-collector exists.

37. user (2026-06-21T13:36:04.486Z)

```

=== web components ===
CompileResult.tsx
IngestPanel.test.tsx
IngestPanel.tsx
LanguageSwitcher.tsx
QueryBar.examples.test.ts
QueryBar.tsx
RallyGrid.tsx
SelectionFooter.tsx
=== grep annotate verbatim (not i18n copy) ===
=== collector curate import-local ===
Already up to date.
src/cli/curate.py:97:     def import_local(self, args):
src/cli/curate.py:656:         (import-local, export, the indexer) keeps running on the ORIGINAL
file.
src/cli/curate.py:666:         # Mirror import-local's video_id derivation so the manifest row
and a
src/cli/curate.py:809:         print(" 3. import-local / export keep pointing at the ORIGINAL
file ? the "
src/cli/curate.py:919:         `import-local`; any blocker issue must be resolved first, since
the

```

```

src/cli/curate.py:957:         print(f"\n {result.n_blockers} BLOCKER issue(s) ? import-
local will reject "
src/cli/curate.py:961:         print(f"      python -m src.cli.curate import-local --sport
<sport> "
src/cli/curate.py:1040:        print("\n GATE: PASS ? review the flagged rallies, then `import-
local`.")
src/cli/curate.py:1239:    import_parser = subparsers.add_parser("import-local", help="Manually
import a local video and annotations")
src/cli/curate.py:1296:                                     help="Manifest key (default: derived from
the filename, like import-local)")

```

38. user (2026-06-21T13:36:04.681Z)

```

10: * NO ``wslCommandMixin`` (no ``wsl -d <distro> bash -c``), NO ``conda activate`` (the
17:   (`WASB_WEIGHTS` / `WASB_REPO_DIR` / `WASB_PYTHON` / `WASB_SCRATCH`), then
68:   python_bin: str = "python"           # the `wasb` conda env python (invoked by path, no
activate)
89:       python_bin=env("WASB_PYTHON") or w.get("python_bin", "python"),
157:   def healthcheck(self) -> Tuple[bool, str]:
175:       return False, f"python binary not found: {self.cfg.python_bin!r} (set
WASB_PYTHON).".
239:       logger.error("python binary not found: %r (set WASB_PYTHON).",
self.cfg.python_bin)

```

39. assistant (2026-06-21T13:36:11.907Z)

I have confirmed the key facts. The SPA has no annotation UI (the grep hits were i18n copy/query terms). The collector `import-local` CLI exists. The native WASB runner needs env vars (WASB_PYTHON/WASB_WEIGHTS/WASB_REPO_DIR) pointing at the separate conda env. Let me do two final verifications: (1) confirm the rally-annotator CSV header line in the actual Lua file, and (2) check the regression.py product-side gate that owns final admission, plus whether there's a per_user training entry. Let me also verify weights.json is gitignored (so a packaged app can't ship a real trained model by default).

40. user (2026-06-21T13:36:13.460Z)

```

12:     rally_number,start_time,end_time,ending_reason,sport,shots_count
105:     .. "(rally_number,start_time,end_time,ending_reason,sport,shots_count; decimal
seconds). "
128:local HEADER = "rally_number,start_time,end_time,ending_reason,sport,shots_count\n"

```

41. user (2026-06-21T13:36:15.056Z)

No matches found

42. user (2026-06-21T13:36:16.235Z)

No matches found

43. assistant (2026-06-21T13:36:19.540Z)

Let me verify the gitignore for weights/artifacts and check the regression.py admission gate more broadly (it's the product-side gate referenced in harness.py:18-19).

44. user (2026-06-21T13:36:21.920Z)

```

=== gitignore weights/artifacts/pth ===
=== regression.py public defs ===
37:def gt_hash(ground_truth: List[metrics.Interval]) -> str:
44:class RegressionResult:
68:def load_baselines(path: str = DEFAULT_BASELINE_PATH) -> Dict[str, dict]:
76:def _baseline_key(video_id: str, stage: str) -> str:
80:def save_baseline(video_id: str, stage: str, precision: float, recall: float,
104:def regress(db: Database, video_id: str, ground_truth: List[metrics.Interval],

```

```
160:def regress_with_provider(db: Database, video_id: str, video_ref: str,
182:def format_result(r: RegressionResult) -> str:
```

45. assistant (2026-06-21T13:36:27.402Z)

The product-side admission gate in regression.py is about performance regression (precision/recall vs baseline), not licensing. The licensing/paid-LLM ship-gate lives entirely in `training/gen0/harness.py::gate\_verdict` + the manifest attestation block. Let me do a final check on the gitignore to confirm weights/model artifacts aren't committed (relevant to packaging ? a packaged app won't carry a trained model), and check how the harness output dir is gitignored.

46. user (2026-06-21T13:36:28.752Z)

```
=== full gitignore (model/data related) ===
Binary file (standard input) matches
```

47. user (2026-06-21T13:36:32.135Z)

```
1      # Byte-compiled / optimized / DLL files
2      __pycache__/
3      *.py[cod]
4      *$py.class
5      *.pt
6
7      # Local environment secrets
8      .env
9      *.env
10     .env.*
11
12     # Provenance stamp baked into a deploy tarball at build time (deploy/gcp/README.md §4);
13     # never a tracked source file ? generated per-deploy, must not ride into a commit.
14     /GIT_SHA
15
16     # C extensions
17     *.so
18
19     # Distribution / packaging
20     .Python
21     build/
22     develop-eggs/
23     dist/
24     downloads/
25     eggs/
26     .eggs/
27     lib/
28     lib64/
29     parts/
30     sdist/
31     var/
32     wheels/
33     share/python-wheels/
34     *.egg-info/
35     .installed.cfg
36     *.egg
37     MANIFEST
38
39     # SQLite databases & Local output files
40     *.db
41     *.db-journal
42     *.db-wal
43     output/
44     output_dir/
45     backend/static/highlights_*.mp4
```

```

46     backend/static/*_compiled.mp4
47
48     # OS generated files
49     .DS_Store
50     Thumbs.db
51     desktop.ini
52
53     # Coverage data
54     .coverage
55     .coverage.*
56     htmlcov/
57     .pytest_cache/
58
59     # Local Claude Code config (machine-specific)
60     .claude/
61     .clocovleerlaage
62     ■
63     ■
64     # Throwaway repro scripts + PR-body / handoff scratch notes (already excluded from
65     # ruff + pytest; never meant to be committed ? keep them out of `git status` too).
66     scratch/
67     .venv/
68

```

48. assistant (2026-06-21T13:36:38.066Z)

I note that `weights.json` is NOT gitignored by pattern, but the harness writes it under `artifacts/<name>/<version>/`, and the docstring says "weights.json (gitignored)". The git status shows `artifacts/` is untracked (in the initial status), and there's a committed `artifacts/rally\_tal\_gen0/0.1.0/manifest.json`. So the manifest is committed but `weights.json` is not present in the tree ? confirming the bundle's weights aren't shipped in the repo. The `\*.pt` is ignored but the WASB `.pth.tar` weights are NOT pattern-ignored (they live in GCS, not the tree).

I have enough to synthesize. Let me record the findings.

49. user (2026-06-21T13:37:46.666Z)

Structured output provided successfully

50. assistant (2026-06-21T13:37:57.096Z)

I've completed the investigation and returned my findings via StructuredOutput. Summary of what I verified against source:

- ****Annotation today is external + VLC-only**** ? the `rally-annotator` MIT VLC 3.0.x Lua plugin in a sibling repo, click-driven, writing `<video>.rallies.csv`. There is NO in-browser or in-engine annotator; the khelsutra design doc locks "link out, don't re-author" (D3) and "no Train now button" (D4).
- ****The CSV bridges to the engine with zero transform**** ? header `rally\_number,start\_time,end\_time,ending\_reason,sport,shots\_count`, read by column name via `gt\_loader.load\_gt\_intervals` and `stub\_runner.synth\_trajectory\_from\_labels`.
- ****Training is an offline CLI**** (`python -m backend.worker train`), with a GPU-free `synth` mock path (turnkey but plumbing-only) and a GPU-gated `detector` path that needs the separate conda/torch-1.11 WASB env + weights.
- ****The licensing ship-gate**** is `training/gen0/harness.py::gate\_verdict` (hard `ship=False` today: C3 + uncalibrated F1) plus the manifest attestation `no\_paid\_llm\_training\_labels:True` / `label\_source: human`.
- ****No serving path consumes a trained bundle yet**** ? `personalization/serving.py` is explicitly not wired in, so "train ? use my model" is not a closed CUJ.

All grounded in the cited file:line references in the structured output.